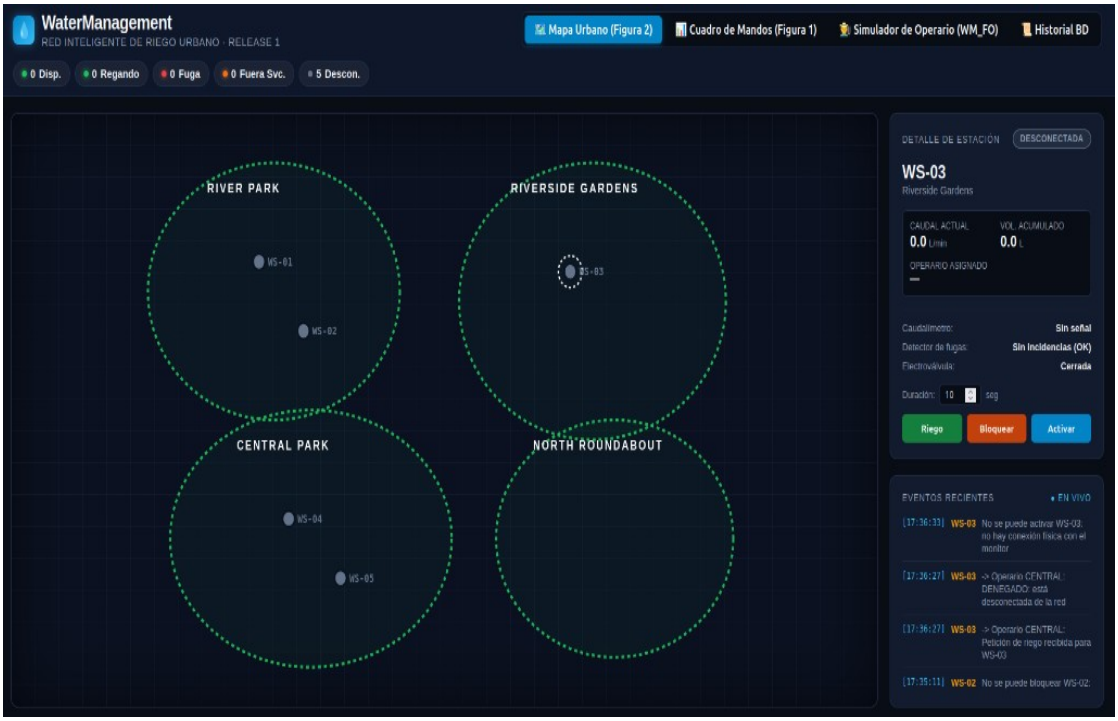
MEMORIA DE PRÁCTICAS: WATERMANAGEMENT NETWORK

Sistemas Distribuidos



Sofía Herrmann Herrero
Katia Hanis

Curso Académico 2026/2027

CONTENIDO

1. Introducción y objetivos · 2. Arquitectura del sistema · 3. Componentes software

4. Protocolos y persistencia · 5. Diseño y resiliencia · 6. Despliegue

7. Validación siguiendo la guía de corrección (pruebas 1–22) · 8. Conclusiones

1. Introducción y objetivos de la práctica

El presente documento recoge el diseño, la implementación, el despliegue y las pruebas de validación del sistema WaterManagement, desarrollado para la primera entrega de la asignatura de Sistemas Distribuidos. El propósito de este proyecto es simular una plataforma distribuida para la gestión, supervisión y control en tiempo real de la red de riego urbano de una ciudad, abarcando distintos parques y zonas verdes.

En un entorno urbano real, el riego manual o sin supervisión centralizada provoca importantes ineficiencias: desde averías o fugas de agua que tardan horas en detectarse hasta la imposibilidad de conocer desde el centro de control qué zonas están regando en cada instante o qué consumo acumulado llevan. Para dar solución a este escenario, WaterManagement plantea una arquitectura distribuida en la que todos sus componentes son programas independientes que se comunican a través de la red sin necesidad de ejecutarse en la misma máquina física. De esta manera, el sistema permite la interacción simultánea de dos tipos de usuarios: el gestor de la red (el ayuntamiento, a través del módulo central) y los operarios de mantenimiento que trabajan sobre el terreno activando y supervisando los riegos mediante su propia aplicación.

Desde el punto de vista técnico, el objetivo principal de la práctica consiste en integrar y comparar dos paradigmas de comunicación fundamentales en los sistemas distribuidos actuales:

- **Comunicación directa mediante Sockets TCP:** empleada para los enlaces de control, registro, autenticación y comprobación periódica del estado de salud de las estaciones, definiendo sobre el flujo de transporte un protocolo propio de empaquetado de tramas a nivel de aplicación con detección de errores.
- **Comunicación asíncrona orientada a eventos mediante Apache Kafka:** utilizada para desacoplar las solicitudes de los operarios, el envío de órdenes hacia los actuadores de las estaciones y la transmisión continua de telemetría de caudal y volumen en tiempo real.

Adicionalmente, el sistema incorpora persistencia relacional mediante una base de datos SQLite gestionada por el nodo central y se empaqueta íntegramente en contenedores Docker para garantizar su portabilidad y permitir su despliegue distribuido entre un entorno Cloud AWS y los ordenadores del laboratorio.

2. Arquitectura general del sistema

La arquitectura de WaterManagement se organiza en tres grandes subsistemas que colaboran a través de la red, siguiendo el modelo conceptual establecido en la especificación de la práctica:

- **Subsistema Central (Core System):** formado por el proceso principal WM_Central y la base de datos relacional SQLite (central.db). Actúa como el punto de coordinación y gobierno de toda la red: autentica las estaciones cuando se conectan, valida las peticiones de los operarios, autoriza o deniega los servicios de riego, registra el histórico de consumos y ofrece tanto un panel de control por consola como una interfaz web en tiempo real.
- **Estaciones de Riego (Watering Stations, WS):** cada estación instalada en un parque o jardín cuenta con un identificador único (por ejemplo, WS-01) y simula tres dispositivos físicos: un caudalímetro (que mide el caudal instantáneo suministrado en litros por minuto), un detector de fugas (que alerta ante pérdidas anómalas o fallos en la instalación) y el actuador de la electroválvula (que abre o cierra el paso del agua). A nivel de software, cada estación se descompone en dos procesos independientes comunicados entre sí por sockets: el Monitor (WM_WS_M), encargado de vigilar la salud del hardware y mantener el enlace con la central, y el Motor o Engine (WM_WS_E), encargado de accionar la electroválvula y calcular el consumo.
- **Operarios de Campo (Field Operators, FO):** representados por la aplicación cliente WM_FO, que permite al personal de mantenimiento consultar el estado de las estaciones disponibles en la ciudad, solicitar la activación del riego en una estación concreta durante un tiempo determinado y hacer el seguimiento en directo del volumen suministrado hasta la finalización del servicio.

La infraestructura de mensajería (Kafka) y la persistencia (SQLite) completan el sistema: Kafka proporciona la mensajería entre los módulos y SQLite almacena la información persistente de las estaciones y las operaciones realizadas.

2.1. Estados operativos de una estación de riego

Para reflejar con exactitud la situación de cada punto de la red, una estación de riego puede encontrarse en cualquiera de los cinco estados siguientes, que se representan tanto en la consola de la central como en el mapa interactivo del panel web mediante un código de colores específico:

- **Disponible (verde):** la estación está conectada a CENTRAL, tanto su proceso Monitor como su proceso Engine funcionan sin incidencias y se encuentra en reposo, lista para atender una solicitud de riego.
- **Regando (verde parpadeante):** la estación funciona correctamente y tiene su electroválvula abierta suministrando agua en ese instante. Mientras permanece en este estado, CENTRAL muestra en tiempo real el caudal instantáneo en litros/minuto, el volumen acumulado en litros y el identificador del operario que solicitó el servicio.
- **Fuera de servicio (naranja):** la estación no sufre ninguna avería física, pero ha sido bloqueada deliberadamente por orden directa de CENTRAL. Si estaba regando en el momento del bloqueo, cierra su electroválvula de inmediato y rechaza cualquier nueva petición hasta que CENTRAL vuelva a activarla.
- **Fuga o avería (rojo):** el Monitor de la estación mantiene la conexión con CENTRAL, pero ha detectado un fallo en el sensor de fuga o ha perdido la comunicación con el

proceso Engine. Por seguridad, si había un riego en curso se interrumpe al instante y la estación queda inhabilitada hasta que la incidencia se resuelve.

- **Desconectada (gris):** CENTRAL no tiene conexión activa con el Monitor de la estación (ya sea porque aún no se ha iniciado o porque se ha caído el proceso o la red). En esta situación CENTRAL desconoce el estado real de la estación y no permite operarla.

El estado comunicado a CENTRAL depende de la disponibilidad del Monitor y del Engine:

Monitor	Engine	Estado
Operativo	Operativo	Disponible (verde)
Operativo	No operativo	Fuga (rojo)
No operativo	Operativo	Desconectada (gris)
No operativo	No operativo	Desconectada (gris)

Cuando WM_Central deja de recibir los mensajes periódicos del Monitor, considera que la estación está desconectada; por tanto, la ausencia de comunicación con el Monitor tiene prioridad sobre el estado del Engine. Si el Monitor continúa operativo pero detecta que el Engine no responde, comunica la incidencia y la estación aparece en estado FUGA. El bloqueo impuesto desde Central se mantiene hasta recibir una orden explícita de activación, y la activación tiene en cuenta el estado real de los componentes: una estación sin Engine operativo no pasa a DISPONIBLE.

3. Descripción de los componentes software implementados

Toda la solución se ha desarrollado en lenguaje Python (versión 3.12), estructurando el código en cinco ficheros principales junto con la plantilla de la interfaz web y los ficheros de configuración para su despliegue en Docker.

3.1. Módulo común de comunicaciones (common.py)

Con el fin de modularizar el código y evitar duplicidades entre los cuatro ejecutables del sistema, hemos creado el módulo `common.py`. Este archivo reúne la implementación a bajo nivel del protocolo de sockets (las funciones de saludo inicial `client_handshake` y `server_handshake`, el cálculo del byte de redundancia longitudinal `lrc` mediante la operación XOR byte a byte, y las funciones `send_frame` y `recv_frame` para enviar y recibir tramas verificadas con confirmación ACK/NACK). Asimismo, define las constantes de estado, las secuencias de escape ANSI para colorear la salida en terminal, los nombres de los tópicos de Apache Kafka y las funciones auxiliares para instanciar productores y consumidores JSON con reintentos de conexión.

3.2. Módulo Central (WM_Central.py y dashboard.html)

El archivo `WM_Central.py` implementa la lógica de gobierno de toda la red mediante una arquitectura concurrente basada en múltiples hilos de ejecución sincronizados con un cerrojo reentrante (`threading.RLock`) para proteger el diccionario de estado de las estaciones y los accesos a la base de datos:

- **Servidor concurrente de Sockets TCP:** escucha en el puerto indicado por línea de comandos (por defecto 5000) y lanza un hilo independiente (_handle_station_socket) por cada Monitor que establece conexión. En este hilo procesa las tramas de alta y autenticación (REGISTRO), actualiza la marca de actividad ante cada latido (PING) con un tiempo máximo de espera de 15 segundos, y gestiona las alertas de avería (LEAK) y recuperación (FIXED). Si el socket se cierra o vence el timeout sin recibir latidos, el bloque finally limpia la sesión, notifica al operario si había un riego activo y pasa la estación automáticamente a estado Desconectada.
- **Consumidores y publicadores de Apache Kafka:** cuenta con tres hilos dedicados a Kafka. El primero escucha en el tópico fo_requests las solicitudes de riego de los operarios; el segundo escucha en ws_events las confirmaciones de apertura de válvula, la telemetría cada segundo y los avisos de fin de riego procedentes de los motores; y el tercero publica cada 2 segundos en el tópico ws_status una instantánea completa del estado de la red.
- **Consola interactiva y Panel Web (dashboard.html):** desde la propia terminal de WM_Central, el administrador dispone de comandos para imprimir la tabla de estado con colores (p), ordenar un riego manual en una estación o en todas a la vez (s <WS|ALL> [segundos]), bloquear estaciones poniéndolas fuera de servicio (b <WS|ALL>), reactivarlas (a <WS|ALL>) y consultar los últimos riegos archivados en SQLite (h). En paralelo, levanta un servidor HTTP multihilo en el puerto 8080 que sirve la aplicación web dashboard.html, ofreciendo en tiempo real un mapa vectorial de los parques de la ciudad, la tabla de monitorización con identificador, ubicación, estado y volumen de agua de cada estación, el estado de los sensores y la consola de eventos. El panel permite además bloquear, activar y regar estaciones de forma individual o global, y dispone de un mecanismo de administración para registrar nuevas estaciones en caliente.

3.3. Monitor de la estación de riego (WM_WS_M.py)

El proceso WM_WS_M.py actúa como vigilante local de la estación y puente de control hacia la central. Recibe como parámetros el puerto local en el que escuchará a su Engine, la dirección IP y puerto de WM_Central y el identificador único de la estación (junto con su ubicación opcional mediante el argumento --loc). En su bucle principal realiza dos tareas de forma continua:

- **Enlace con WM_Central:** mediante el método ensure_central_connection, establece conexión TCP con el servidor central, ejecuta el handshake inicial y envía la trama REGISTRO#<ID>#<UBICACION>. Si la central se encuentra caída o se reinicia en mitad de la ejecución, el Monitor captura la excepción de red y reintenta la reconexión automáticamente en cada ciclo, enviando además un latido PING#<ID> cada 3 segundos mientras la conexión permanece activa.
- **Supervisión del Engine:** abre un socket servidor local en espera de que se conecte el proceso WM_WS_E. En cuanto el Engine se conecta, el Monitor le transfiere dinámicamente su identidad enviándole la trama ASSIGN#<ID>. A partir de ese momento, cada segundo le envía la trama de comprobación HEALTH. Si el Engine responde OK, el Monitor notifica FIXED#<ID> a la central para que la estación quede

Disponible; si el Engine no se ha conectado aún, cierra la conexión o responde KO, el Monitor envía de inmediato LEAK#<ID> a la central para declararla en estado de Fuga (motivo NO_ENGINE cuando el Engine deja de responder).

3.4. Motor y actuador de la estación (WM_WS_E.py)

El proceso WM_WS_E.py simula los elementos físicos de riego. Al iniciarse únicamente conoce la dirección del broker Kafka y la dirección del socket de su Monitor, por lo que no tiene ningún identificador de estación fijado en código. Su primer paso consiste en conectarse por socket al Monitor y esperar en un evento de sincronización (assigned_event) hasta recibir la trama ASSIGN#<ID>. Una vez conoce su identidad, lanza el hilo consumidor de Kafka sobre el tópico ws_commands filtrando las órdenes dirigidas a su estación:

- Ciclo de riego y telemetría: al recibir START (o al activarse manualmente), el Engine comprueba que la estación pueda prestar servicio, abre la electroválvula simulada y activa inmediatamente el temporizador independiente de Kafka. La publicación de STARTED se realiza en segundo plano, sin bloquear el control local ni las respuestas HEALTH del Monitor. El hilo de riego genera caudal y acumula volumen cada segundo; la telemetría solo se publica cuando el intento de STARTED ha concluido correctamente. Al finalizar, la electroválvula se cierra antes de cualquier espera de red, se guarda el resumen y se publica FINISHED respetando el orden de eventos. Los resúmenes que no pueden enviarse quedan pendientes para reintento cuando se recupera Kafka. El control del temporizador fue corregido para evitar duraciones superiores a las solicitadas.
- **Simulación interactiva de averías y control local:** la consola del Engine ofrece un menú en tiempo real donde el usuario puede pulsar la tecla k para conmutar el estado del sensor de fugas entre normal (OK) y averiado (KO). Si se pulsa k mientras la estación está regando, el propio Engine cierra la electroválvula inmediatamente con motivo LEAK y pasa a responder KO al sondeo HEALTH del Monitor. Asimismo, permite iniciar un riego manual pulsando s y detenerlo pulsando f (motivo MANUAL).

3.5. Aplicación del operario de campo (WM_FO.py)

El programa WM_FO.py proporciona la interfaz de trabajo para los jardineros y operarios de mantenimiento. Al arrancar recibe la dirección del broker Kafka y el identificador del operario (por ejemplo, FO-01). Lanza un hilo consumidor en segundo plano suscrito a los tópicos ws_status (para mantener actualizada la lista de estaciones de la red) y fo_replies (filtrando exclusivamente los mensajes cuyo campo op coincide con el identificador del operario).

En modo interactivo permite listar el estado y caudal de todas las estaciones (comando l), solicitar un riego (comando r <WS> [seg]) y finalizar la aplicación (q), mostrando por pantalla cada paso de validación recibido desde CENTRAL, la telemetría en directo segundo a segundo y el cuadro resumen final al concluir el servicio. Además, implementa el modo de ejecución por lotes mediante el parámetro --file operaciones.txt: lee secuencialmente las líneas del fichero (con formato WS-ID,segundos), envía la primera solicitud y bloquea el avance mediante un evento (service_done_event) hasta recibir el mensaje SUMMARY o DENY de CENTRAL. Una

vez concluido cada servicio, espera estrictamente los 4 segundos exigidos en el enunciado antes de enviar la siguiente petición.

4. Protocolos de comunicación y persistencia de datos

4.1. Protocolo de tramas sobre Sockets TCP (<STX><DATA><ETX><LRC>)

Dado que el protocolo TCP ofrece un servicio de transporte orientado a flujo de bytes (stream) sin conservar las fronteras entre los mensajes enviados, ha sido necesario definir un protocolo de nivel de aplicación entre el Monitor y CENTRAL, así como entre el Monitor y el Engine. Siguiendo la recomendación de la especificación, hemos implementado el estándar de empaquetado <STX><DATA><ETX><LRC> con el siguiente funcionamiento:

- **Establecimiento de sesión (Handshake):** nada más abrir el socket TCP, el cliente transmite un byte de consulta ENQ (0x05) y aguarda a que el servidor responda con un byte de confirmación ACK (0x06) antes de enviar datos.
- **Estructura de la trama:** cada mensaje se codifica en UTF-8 separando la operación y sus parámetros mediante el carácter almohadilla (por ejemplo, REGISTRO#WS-01#River Park Norte). El bloque de datos se precede del byte de control STX (0x02, Start of Text) y se cierra con el byte ETX (0x03, End of Text). A continuación se adjunta un byte LRC (Longitudinal Redundancy Check), calculado como la suma lógica exclusiva (XOR) acumulada byte a byte sobre el contenido de DATA.
- **Control de errores y retransmisión:** el receptor lee del socket hasta encontrar STX, acumula los bytes hasta ETX y lee el byte siguiente como LRC esperado. Si el LRC calculado localmente coincide con el recibido, devuelve un byte ACK (0x06) y entrega el mensaje a la aplicación; en caso contrario, devuelve un byte NACK (0x15) y el emisor retransmite automáticamente la trama hasta un máximo de 3 intentos.

4.2. Arquitectura de eventos en Apache Kafka

Para la comunicación asíncrona a través de Apache Kafka hemos estructurado el flujo de mensajes JSON en cinco tópicos diferenciados, que WM_Central verifica y crea automáticamente al arrancar mediante KafkaAdminClient:

- **fo_requests:** producido por WM_FO y consumido por WM_Central. Transporta las solicitudes de activación de riego indicando el identificador del operario, la estación objetivo, la duración en segundos y la marca de tiempo.
- **fo_replies:** producido por WM_Central y consumido por WM_FO. Envía a cada operario los mensajes de progreso (STEP), la autorización de inicio (AUTH), la denegación motivada (DENY), la telemetría en vivo (TELEMETRY) y el informe final del servicio (SUMMARY).
- **ws_commands:** producido por WM_Central y consumido por WM_WS_E. Canaliza las órdenes de control hacia los motores de las estaciones (START, STOP, BLOCK y ACTIVATE).
- **ws_events:** producido por WM_WS_E y consumido por WM_Central. Permite a los motores notificar la apertura efectiva de la electroválvula (STARTED), las lecturas de

caudal y volumen cada segundo (TELEMETRY) y el cierre final del suministro (FINISHED).
- **ws_status:** publicado periódicamente por WM_Central y consumido por todas las instancias de WM_FO para conocer el listado global de estaciones y su disponibilidad.

4.3. Base de datos relacional SQLite (central.db)

El módulo WM_Central mantiene la persistencia del sistema en el fichero SQLite central.db a través de tres tablas relacionales:

- **operators:** almacena el identificador y nombre de los operarios autorizados, precargando de FO-01 a FO-05.
- stations: registra el identificador, ubicación y estado de cada estación. Se precargan WS-01 a WS-05 y pueden incorporarse nuevas estaciones mediante el mecanismo de alta administrativa, como WS-06 durante el desarrollo y WS-07 durante la validación. El registro nuevo se conserva en SQLite y no sustituye indebidamente los datos de una estación ya registrada.
- **watering_history:** tabla de auditoría donde se inserta cada servicio finalizado con su marca temporal de inicio y fin, operario (incluyendo MANUAL para el riego desde el menú del Engine y CENTRAL para el riego desde el panel), estación, volumen total en litros, duración en segundos, causa de cierre (TIME, LEAK, MANUAL, CENTRAL_BLOCK, MONITOR_DOWN) y un identificador único de transacción (transaction_id).

La base de datos se almacena en el directorio persistente /data, asociado a un volumen Docker. El registro administrativo de nuevas estaciones valida su identificador y evita modificar por error la ubicación o el estado de una estación que ya está dada de alta.

5. Justificación de las decisiones de diseño y resiliencia

A lo largo del desarrollo y depuración del sistema hemos adoptado

1. **Elección híbrida de Sockets TCP y Apache Kafka:** utilizar Sockets TCP entre el Monitor y CENTRAL (y entre el Monitor y el Engine) resulta idóneo porque la supervisión de salud requiere una sesión punto a punto continua donde la caída de un extremo o el corte de la conexión pueda detectarse en pocos segundos (mediante los sondeos HEALTH cada segundo y los latidos PING cada 3 segundos). Por el contrario, emplear Apache Kafka para comunicar a los operarios con CENTRAL y con los motores permite desacoplar espacial y temporalmente a los participantes: CENTRAL no necesita conocer las direcciones IP de los operarios ni abrir sockets hacia los motores, pueden incorporarse nuevas instancias de operarios o estaciones en cualquier momento, y el flujo continuo de telemetría por segundo se distribuye sin bloquear los hilos de control.
2. **Desacoplamiento interno de la estación en Monitor y Engine:** separar cada estación en dos procesos independientes (WM_WS_M y WM_WS_E) garantiza que un fallo o cierre abrupto del motor (por ejemplo, al detener el proceso con Ctrl+C) no deje a la estación sin capacidad de reportar la incidencia. Aunque el Engine caiga, el Monitor sigue en ejecución, detecta en el siguiente segundo que el socket local no responde al

mensaje HEALTH y transmite de inmediato la trama LEAK#<ID> a CENTRAL para abortar el servicio y marcar la estación en rojo. Además, al hacer que el Monitor asigne el ID al Engine por socket (ASSIGN#<ID>), evitamos duplicar identificadores en la configuración.

3. **Registro inicial en estado de Fuga hasta verificar la conexión del Engine:** cuando el Monitor de una estación se conecta y se autentica en CENTRAL, todavía no existe garantía de que su proceso Engine haya sido arrancado. Si CENTRAL marcase la estación como Disponible nada más conectar el Monitor, un operario podría solicitar un riego sobre una estación que carece de motor para abrir la válvula. Por ello, el Monitor arranca con leak_active = True por defecto y CENTRAL registra toda estación recién autenticada en estado FUGA, pasando a DISPONIBLE únicamente cuando el Engine se conecta al Monitor, supera el chequeo HEALTH y el Monitor envía la trama FIXED.
4. **Confirmación en dos pasos al iniciar el riego (pending_start y timeout de 10 segundos):** cuando CENTRAL valida una petición de riego, no cambia el estado de la estación a REGANDO ni envía la autorización definitiva (AUTH) en el mismo instante de publicar la orden START en Kafka, ya que el Engine podría haber sufrido un fallo justo en ese momento. En su lugar, CENTRAL activa la bandera pending_start = True y lanza un hilo temporizador de 10 segundos (check_watering_timeout). Solo cuando el Engine abre efectivamente la electroválvula y devuelve el evento STARTED por Kafka, CENTRAL cambia el estado a REGANDO y notifica AUTH al operario; si transcurren 10 segundos sin confirmación del Engine, CENTRAL cancela la reserva y notifica la denegación (DENY) al operario. Una segunda solicitud recibida mientras la primera sigue pendiente se deniega.
5. Recepción de mensajes al iniciar consumidores Kafka mediante offset earliest y filtrado temporal: los consumidores usan identificadores de grupo únicos y auto_offset_reset='earliest', descartando los mensajes cuya marca temporal sea anterior al arranque. Esto reduce la posibilidad de perder las primeras respuestas al iniciar los consumidores, sin implicar que Kafka pueda entregar toda la telemetría generada mientras está indisponible.
6. **Reinicio seguro de CENTRAL con estado Desconectada:** en cumplimiento de la primera regla de negocio del enunciado, cada vez que WM_Central se inicia ejecuta una sentencia UPDATE sobre la tabla stations de SQLite fijando el estado de todas las estaciones conocidas a DESCONECTADA. De este modo, si CENTRAL se reinicia tras una caída, nunca muestra estados antiguos como válidos hasta que los monitores restablecen la conexión por socket.
7. **Identificación única de transacciones y prevención de duplicados:** cada operación de riego lleva un identificador único (transaction_id) que evita que un mismo evento de finalización se registre varias veces en SQLite, incluso cuando Kafka entrega mensajes repetidos. Central registra la transacción antes de enviar el resumen al operario y confirma los mensajes de Kafka después de procesarlos correctamente, lo que permite recuperar operaciones interrumpidas y conservar sus resultados.
8. **Bloqueo persistente y activación consciente del estado real:** el bloqueo impuesto desde CENTRAL se mantiene aunque el Engine se averié o se reinicie, y solo una orden explícita de activación devuelve la estación a servicio. La activación (individual o

global) se procesa estación a estación y tiene en cuenta el estado real de sus componentes: una estación sin Engine vuelve a FUGA y una desconectada permanece desconectada, de modo que nunca se fuerza a Disponible una estación que no está operativa.
Independencia frente a la caída de Kafka: los intentos de publicación de eventos se realizan fuera de los bloqueos utilizados para responder al Monitor; el temporizador y el cierre de la electroválvula no dependen de la disponibilidad del broker. Los resúmenes FINISHED fallidos se mantienen pendientes y se reenvían al restablecerse la comunicación.

9. **Política de seguridad ante la pérdida de componentes durante un riego:** si el Engine pierde la conexión con su Monitor, cierra la electroválvula de inmediato con motivo `MONITOR_DOWN`; si es el Engine el que cae, el Monitor notifica la avería y el riego finaliza con motivo `LEAK`. En cambio, si caen el Front Office o WM_Central, el Engine completa el riego solicitado y el resultado se entrega o notifica cuando el componente se recupera.

6. Guía de despliegue y puesta en marcha

Desplegar una aplicación distribuida consiste en poner en funcionamiento cada uno de sus componentes en su entorno de ejecución para que puedan comunicarse a través de la red. Cumpliendo el requisito de la práctica, todos los módulos se despliegan utilizando contenedores Docker sin necesidad de emplear ningún entorno de desarrollo (IDE) ni modificar el código fuente para parametrizar direcciones IP, puertos o identificadores.

6.1. Despliegue de WM_Central en AWS

WM_Central se ejecuta en una instancia de AWS mediante Docker. El contenedor publica el puerto 5000 para la comunicación con los demás módulos y el puerto 8080 para el panel web. Desde el directorio raíz del proyecto se construye la imagen:

```
sudo docker build -f Dockerfile -t wm_central_aws:latest .
```

A continuación, se inicia el contenedor:

```
sudo docker run -d \
  --name wm_central \
  --restart unless-stopped \
  -p 5000:5000 \
  -p 8080:8080 \
  -v /home/ubuntu/Practica-1-SD/data:/data \
  -e DB_PATH=/data/central.db \
  wm_central_aws:latest \
  python WM_Central/WM_Central.py \
  5000 172.31.32.90:19092 \
  --web-port 8080 \
  --db /data/central.db
```

El panel de supervisión se consulta desde un navegador mediante http://<IP_PUBLICA_AWS>:8080/. La base de datos SQLite se almacena en el directorio persistente /data, asociado a un volumen Docker.

Para el desarrollo local, el fichero docker-compose.yml define el contenedor del broker Apache Kafka (en modo KRaft, sin necesidad de ZooKeeper) y el contenedor de WM_Central, montando un volumen persistente (./data:/data) para conservar central.db:

```
docker compose up -d
docker compose logs --tail=30 central
```

En ese caso, el servidor de sockets de CENTRAL queda a la escucha en el puerto 5000, el broker de Kafka en el puerto 9092 y el cuadro de mandos web en http://localhost:8080.

6.2. Ejecución de las estaciones WM_WS

Cada estación dispone de un identificador y una ubicación propios. Su Monitor se conecta con WM_Central y comunica periódicamente el estado de la estación; el Engine se ejecuta como parte de la estación y se encarga del control del riego, la electroválvula y la generación de telemetría. Para facilitar el lanzamiento de tantas instancias como se requieran durante la evaluación, hemos creado el script docker_run.sh, que ejecuta la imagen Docker wm_app pasando todos los argumentos por línea de comandos.

Ejemplo de ejecución del Monitor de WS-07 (puerto local 6007, CENTRAL en AWS):

```
./docker_run.sh monitor 6007 13.51.207.62:5000 WS-07 --loc "Central Park Oeste"
```

Ejemplo de ejecución del Engine, conectado al broker Kafka y al Monitor local:

```
./docker_run.sh engine 16.171.14.87:9092 localhost:6007
```

6.3. Ejecución de WM_FO

Cada operador se identifica mediante un identificador propio y se conecta al broker Kafka configurado para el sistema:

```
./docker_run.sh fo 16.171.14.87:9092 FO-03
```

Los principales comandos disponibles en la consola del operador son:

Comando	Función
l	Consultar las estaciones y sus estados.
r <WS> [seg]	Solicitar un riego para una estación durante el tiempo indicado (por ejemplo, r WS-07 5).
q	Finalizar la ejecución del Front Office.

El operario también puede ejecutarse en modo automático por lotes (espera de 4 s entre servicios):

```
./docker_run.sh fo 16.171.14.87:9092 FO-01 --file operaciones.txt
```

En el escenario de despliegue distribuido, con la parte central alojada en la nube (AWS) y el resto de módulos repartidos en distintos ordenadores del laboratorio, basta con sustituir las

direcciones locales por la dirección IP pública de la instancia o el endpoint del proxy TCP proporcionado por la plataforma.

6.4. Preparación del entorno y del repositorio

Para reproducir el despliegue utilizado en las pruebas, partimos de dos instancias AWS: una ejecuta WM_Central con SQLite y el panel web; la otra aloja el broker Kafka. Los Monitores y Engines de las estaciones se arrancan en un ordenador del laboratorio, y los operarios de campo, en el segundo. La distribución puede variar si el profesor solicita otras instancias.

Antes de empezar comprobamos que las instancias de AWS están encendidas, que disponemos de conexión de red y que Docker está operativo en los dos ordenadores. Accedemos al repositorio del proyecto y utilizamos su rama main para trabajar con el mismo código:

```
git clone https://github.com/shh14-ua/Practica-1-SD.git
cd Practica-1-SD
```

Si el proyecto ya está descargado y no contiene cambios locales pendientes, lo actualizamos con git pull --ff-only origin main. Para la evaluación deben comprobarse también la estructura del repositorio exigida por el enunciado (WM_Central, WM_WS, WM_FO, prompts, README.md y Nombres.txt) y el acceso concedido al profesor. La existencia de Nombres.txt y los permisos del repositorio deben verificarse antes de la entrega.

6.5. Comprobación y arranque de los servicios AWS

El día de la evaluación, la parte cloud debe estar ya desplegada. En la instancia AWS de Kafka, accedemos por SSH y comprobamos el contenedor. Solo si está detenido lo iniciamos:

```
sudo docker ps --filter name=kafka
sudo docker start kafka
```

En la instancia AWS de Central realizamos la misma comprobación, consultamos los registros y abrimos el panel web:

```
sudo docker ps --filter name=wm_central
sudo docker start wm_central
sudo docker logs --tail 40 wm_central
```

Panel web del despliegue utilizado: http://13.51.207.62:8080. El puerto 5000 se utiliza para los sockets TCP de los Monitores y el 8080 para la interfaz web. SQLite se conserva en /data/central.db mediante un volumen persistente. Los comandos docker start se emplean únicamente cuando el contenedor correspondiente no está en ejecución; no es necesario recrear Central ni borrar la base de datos.

6.6. Preparación de Docker en los ordenadores del laboratorio

En cada ordenador entramos en la carpeta del repositorio. Cuando la imagen wm_app no existe, o si hemos actualizado el código, la construimos y comprobamos que está disponible:

```
docker build -t wm_app .
docker images wm_app
```

Para arrancar las instancias empleamos docker_run.sh, que recibe los parámetros de cada componente. El montaje de las estaciones y operadores no requiere abrir un IDE ni modificar el código fuente. Si ya hay instancias antiguas, comprobamos docker ps antes de iniciarlas para evitar duplicidades o puertos ocupados.

6.7. Orden de arranque de una estación de riego

Primero abrimos una terminal para el Monitor. Por ejemplo, para WS-03, que utiliza el puerto local 6003 y se conecta con Central de AWS:

```
./docker_run.sh monitor 6003 13.51.207.62:5000 WS-03 --loc "Riverside Gardens"
```

El Monitor informa de su autenticación y queda a la espera del Engine. Mientras este no esté conectado, Central puede mostrar la estación como FUGA (NO_ENGINE); no debe considerarse disponible todavía. Sin cerrar esa terminal, abrimos una segunda y ejecutamos:

```
./docker_run.sh engine 16.171.14.87:9092 localhost:6003
```

El Engine recibe el identificador WS-03 mediante ASSIGN, responde a HEALTH y el Monitor comunica la recuperación a Central. Verificamos en el panel que WS-03 pasa a DISPONIBLE. Para añadir otras estaciones cambiamos el identificador y el puerto, manteniendo el mismo procedimiento. Las estaciones nuevas, como WS-07, deben registrarse previamente a través del mecanismo de administración de Central; conectar únicamente su Monitor no autoriza el alta.

[CAPTURA 1 PENDIENTE: terminales del Monitor y Engine autenticados y panel con la estación disponible.]

6.8. Arranque de los operarios y riego de comprobación

Con al menos una estación disponible, abrimos otra terminal en el segundo ordenador del laboratorio e iniciamos un operador:

```
./docker_run.sh fo 16.171.14.87:9092 FO-03
```

En su menú escribimos l para consultar los estados. Después solicitamos un servicio de corta duración en una estación operativa:

```
r WS-03 5
```

Comprobamos la recepción de la solicitud, su autorización, la apertura de la electroválvula, la telemetría y el resumen final. El servicio debe terminar automáticamente con motivo TIME y aparecer registrado en SQLite. Podemos iniciar otro operario en una terminal adicional usando otro identificador, sin detener el primero.

[CAPTURA 2 PENDIENTE: FO con solicitud autorizada, telemetría y resumen final; preferiblemente con el panel de Central visible.]

6.9. Verificación final y preparación para las pruebas

Antes de comenzar la guía de corrección comprobamos que Central y Kafka siguen activos en AWS, que los Monitores y Engines están conectados, que el panel muestra los estados previstos y que FO puede consultar y solicitar riegos. Dejamos abiertas las terminales para observar los mensajes al provocar averías, detener componentes o incorporar nuevas instancias. Durante estas pruebas solo se detiene el componente indicado en cada caso.

Como evidencia del despliegue distribuido incorporaremos capturas reales de las aplicaciones conectadas. Las imágenes deben corresponder a la ejecución realizada y mostrar, cuando sea posible, el identificador de estación, el estado y las notificaciones pertinentes.

[CAPTURA 3 PENDIENTE: panel web con varias estaciones y sus estados durante la ejecución distribuida.]

7. Pruebas realizadas, detección de errores y validación final

Las pruebas se organizan según los 22 criterios consecutivos de la Guía de Corrección 2026/2027: despliegue (1–3), funcionamiento base (4–14), resiliencia (15–19) y aspectos generales (20–22). Las comprobaciones de un mismo criterio se agrupan como subpruebas. Cuando se describe una incidencia de una versión preliminar, se distingue expresamente de la validación posterior de la versión corregida.

7.1. Defectos detectados en la versión preliminar y correcciones aplicadas

Durante el desarrollo, las primeras pruebas sobre el entorno Dockerizado nos permitieron detectar defectos en una versión previa del código. Al levantar el entorno con Docker Compose comprobamos que el servidor de sockets se iniciaba en el puerto 5000, se enlazaba con Kafka y habilitaba el panel web en el puerto 8080, y que, al no haberse conectado ningún Monitor, todas las estaciones aparecían DESCONECTADAS (gris) y CENTRAL rechazaba inmediatamente cualquier riego sobre ellas.

A continuación probamos el alta en caliente de una nueva estación (WS-06, ubicación Parque Municipal) arrancando únicamente su Monitor:

```
./docker_run.sh monitor 6006 localhost:5000 WS-06 --loc "Parque Municipal"
```

En aquella primera versión, WS-06 aparecía directamente como DISPONIBLE solo con el Monitor, sin tener el Engine conectado. Al pulsar el botón de regar durante 10 segundos desde el panel de CENTRAL, el riego se daba por iniciado erróneamente: CENTRAL aceptaba la solicitud sin que hubiese ningún Engine para abrir la electroválvula, y la estación permanecía en estado Regando indefinidamente con el caudal y el volumen estancados en 0,0 L, por lo que hubo que detenerla con el botón Bloquear.

Esta prueba permitió identificar cuatro defectos: (1) una estación cuyo Engine no está conectado no puede figurar como Disponible; (2) CENTRAL no debe autorizar ni dar por iniciado un riego sin confirmación de que el Engine ha comenzado realmente a suministrar agua; (3) sin el Engine no se contabilizaba el tiempo ni el volumen; y (4) el indicador de Regando parpadeaba en azul en lugar de verde. Se aplicaron tres correcciones en `WM_WS_M.py`, `WM_Central.py` y `dashboard.html`:

- **Estado inicial en Fuga hasta que el Engine se conecta:** el Monitor inicializa `leak_active = True` y responde `health = 'KO'` mientras no exista conexión con el Engine; CENTRAL registra la estación en estado FUGA al autenticarse el Monitor, y pasa a Disponible cuando llega la trama FIXED. La activación desde CENTRAL solo se permite sobre estaciones fuera de servicio y respeta el estado real de los componentes.
- **Confirmación de apertura del Engine con timeout de 10 segundos:** CENTRAL marca `pending_start = True`, envía START por Kafka y lanza `check_watering_timeout`; solo al recibir STARTED cambia a REGANDO y envía AUTH. Sin confirmación en 10 segundos, cancela la solicitud y envía DENY.
- **Color del estado Regando:** en `dashboard.html` se actualizó la variable CSS `--color-watering` y la regla `.badge-REGANDO` al verde (#22c55e) con la animación de parpadeo `blink-anim`.

Las pruebas siguientes, ya sobre la versión final y desplegadas en AWS y en el laboratorio, validan todos los escenarios de evaluación.

7.2. Pruebas de validación

Prueba 1. Despliegue de WM_Central y servicios de infraestructura

Objetivo: comprobar que WM_Central y los servicios necesarios para el funcionamiento del sistema pueden desplegarse en la infraestructura cloud y permanecer operativos.

Procedimiento:

1. Accedimos mediante SSH a la instancia AWS destinada al despliegue.
2. Construimos la imagen Docker de WM_Central desde el directorio raíz del proyecto.
3. Iniciamos el contenedor publicando los puertos 5000 y 8080.
4. Configuramos la conexión con Kafka y la ruta persistente de SQLite.
5. Comprobamos el estado del contenedor y consultamos sus registros.
6. Accedimos al panel web desde un navegador.

Resultado: WM_Central quedó desplegado en AWS mediante Docker. El contenedor permaneció activo y los puertos de comunicación y del panel web quedaron publicados. El servicio utilizó Kafka como infraestructura de mensajería y SQLite para almacenar la información persistente. El panel web quedó accesible a través del puerto 8080 y permitió consultar las estaciones registradas.

Prueba 2. Ejecución independiente de los módulos y despliegue mediante Docker

Objetivo: comprobar que los módulos del sistema pueden ejecutarse de forma independiente, sin utilizar un entorno de desarrollo, y que la arquitectura permite distribuirlos entre diferentes máquinas.

Procedimiento:

1. Desplegamos WM_Central y Kafka en AWS.
2. Ejecutamos los módulos WM_WS y WM_FO en los ordenadores del laboratorio.
3. Iniciamos los contenedores mediante los scripts de ejecución del proyecto.
4. Comprobamos que los módulos podían conectarse a los servicios centrales sin necesidad de abrir un IDE.
5. Verificamos que cada instancia utilizaba sus propios parámetros de identificación y conexión.

Resultado: los módulos pudieron ejecutarse mediante Docker en máquinas diferentes. WM_Central y Kafka permanecieron alojados en AWS, mientras que los operadores y las estaciones se ejecutaron en los ordenadores del laboratorio. La arquitectura permitió mantener separados los procesos y sus respectivos ciclos de ejecución.

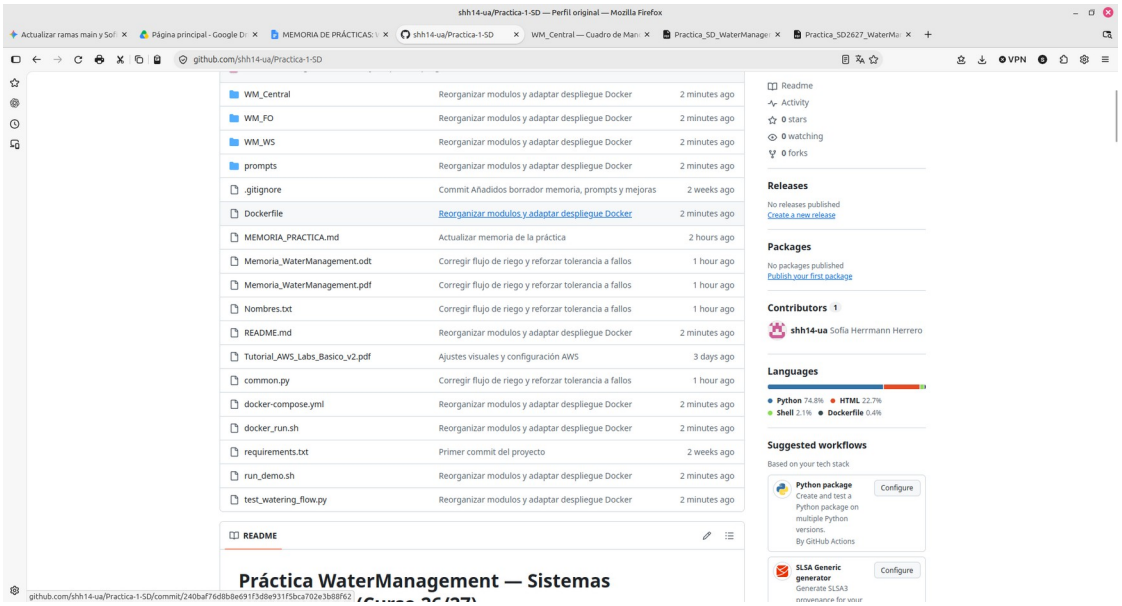


Figura 1. Estructura del repositorio del proyecto, con los módulos WM_Central, WM_FO y WM_WS en directorios independientes y los scripts de despliegue con Docker.

Prueba 3. Despliegue dinámico de múltiples instancias

Objetivo: comprobar que el sistema permite ejecutar simultáneamente varias instancias de WM_FO y WM_WS, así como incorporar, detener y recuperar instancias sin reiniciar el resto del sistema.

3.1. Ejecución simultánea de operadores de campo. En el ordenador LLIBRE1-11 iniciamos dos instancias independientes de WM_FO mediante Docker, identificadas como FO-01 y FO-02:

```
./docker_run.sh fo 16.171.14.87:9092 FO-01
./docker_run.sh fo 16.171.14.87:9092 FO-02
```

Ambas instancias arrancaron correctamente y mostraron sus respectivos menús de operaciones. Desde las dos terminales ejecutamos el comando `l` y los dos operadores recibieron el mismo listado, con WS-01 y WS-02 en estado DISPONIBLE y las demás estaciones desconectadas.

Resultado: varias instancias de WM_FO pueden permanecer activas simultáneamente, conectadas al mismo broker Kafka y sin interferir entre ellas.

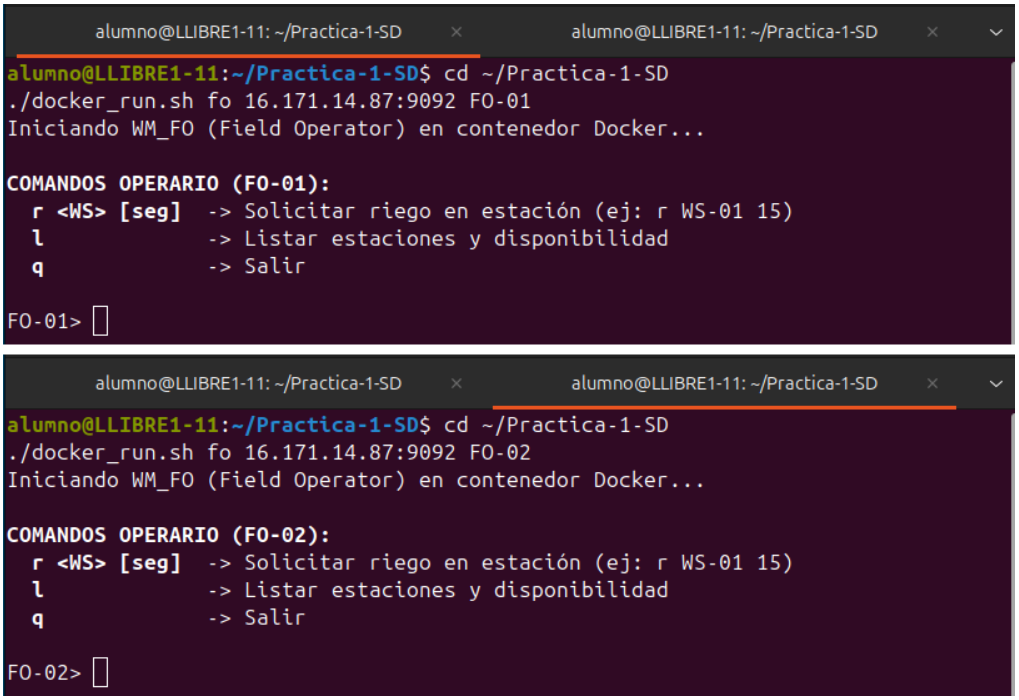


Figura 2. Arranque de los operadores FO-01 y FO-02 mediante Docker en LLIBRE1-11, con sus respectivos menús de operaciones.

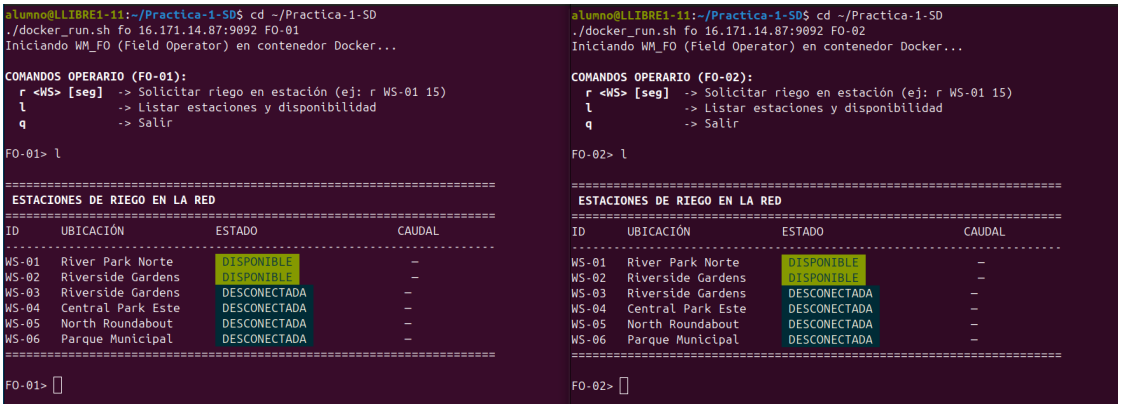


Figura 3. Comando l ejecutado desde FO-01 (izquierda) y FO-02 (derecha): ambos reciben el mismo listado, con WS-01 y WS-02 DISPONIBLE y el resto DESCONECTADA.

3.2. Arranque simultáneo de varias estaciones. En el despliegue distribuido iniciamos WS-01 y WS-02, cada una formada por un Monitor y un Engine. Los Monitors se conectaron y autenticaron correctamente en WM_Central, mientras que los Engines recibieron sus identificadores correspondientes. Comprobamos en el dashboard que ambas estaciones aparecían en estado DISPONIBLE.

Resultado: el arranque simultáneo de las dos estaciones se realizó correctamente y ambas quedaron registradas y disponibles en WM_Central.

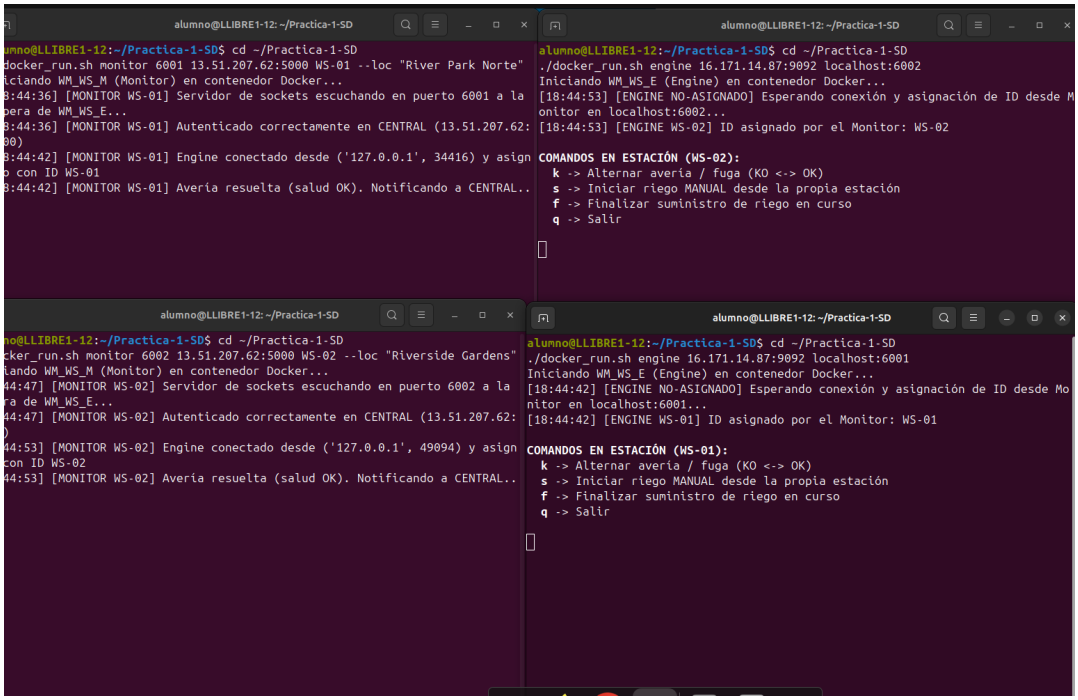


Figura 4. Monitors y Engines de WS-01 y WS-02 en LLIBRE1-12: autenticación en CENTRAL y asignación de los identificadores WS-01 y WS-02.

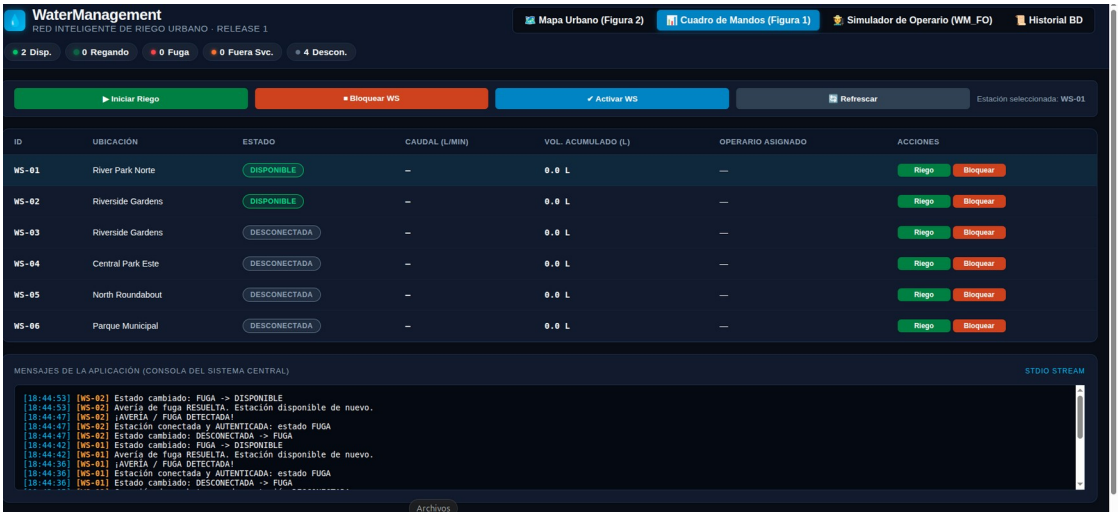


Figura 5. Panel de WM_Central con WS-01 y WS-02 en estado DISPONIBLE y el resto de estaciones DESCONECTADA.

3.3. Recuperación de un operador tras una caída. Después de detener bruscamente FO-03 mediante docker kill, volvimos a iniciar el operador desde una terminal de LLIBRE1-11:

```
./docker_run.sh fo 16.171.14.87:9092 FO-03
```

Una vez iniciado, ejecutamos l. Durante la operación mantuvimos en funcionamiento WM_Central, Kafka, los otros operadores y las estaciones.

Resultado: FO-03 arrancó correctamente y recibió el listado de las seis estaciones registradas. WS-01, WS-02 y WS-03 aparecieron como DISPONIBLE, mientras que las

restantes figuraban como DESCONECTADA. No fue necesario reiniciar los demás componentes.

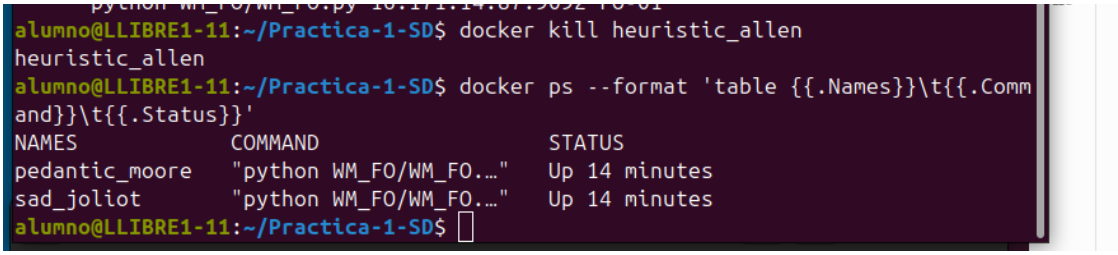


Figura 6. Detención brusca de un operador con docker kill; docker ps muestra que los otros dos operadores siguen en ejecución.



Figura 7. Listados de estaciones consultados desde FO-03: WS-03 pasa de DESCONECTADA a DISPONIBLE.

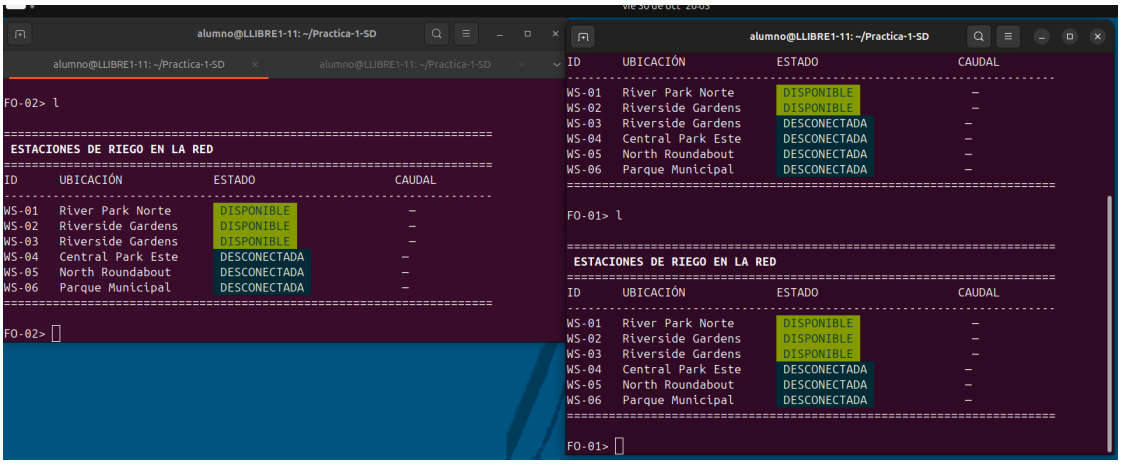


Figura 8. FO-02 (izquierda) y FO-01 (derecha) consultando el listado de estaciones, con WS-01, WS-02 y WS-03 DISPONIBLE.

3.4. Caída repentina y recuperación de una estación. En LLIBRE1-12 teníamos tres estaciones funcionando simultáneamente: WS-01, WS-02 y WS-03. Identificamos los contenedores de Monitor y Engine de WS-03 mediante:

```
docker ps --no-trunc --format 'table {{.ID}}\t{{.Names}}\t{{.Command}}'
```

A continuación, simulamos la caída repentina de ambos componentes y, tras consultar las estaciones desde FO-03, recuperamos WS-03:

```
docker kill magical_johnson jovial_almeida
./docker_run.sh monitor 6003 13.51.207.62:5000 WS-03 --loc "Riverside Gardens"
./docker_run.sh engine 16.171.14.87:9092 localhost:6003
```

Resultado: al detener WS-03, sus estaciones compañeras WS-01 y WS-02 continuaron funcionando. WM_Central detectó la caída y actualizó WS-03 a DESCONECTADA. Tras reiniciar el Monitor y el Engine, ambos se conectaron correctamente y las tres estaciones aparecieron nuevamente como DISPONIBLE.

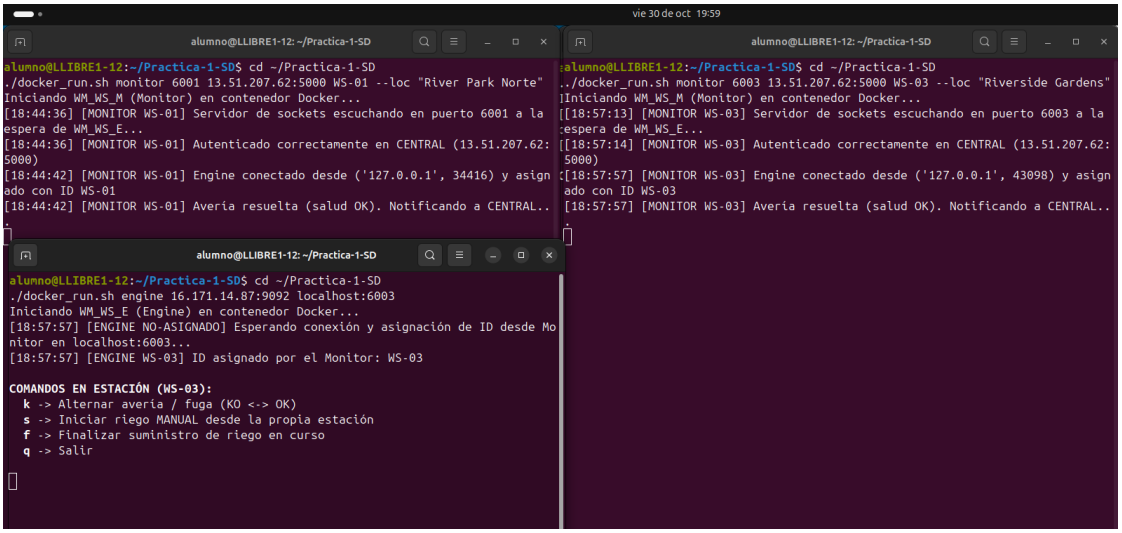


Figura 9. Monitor y Engine de WS-03 en LLIBRE1-12: el Monitor se autentica en CENTRAL y el Engine recibe el identificador WS-03.

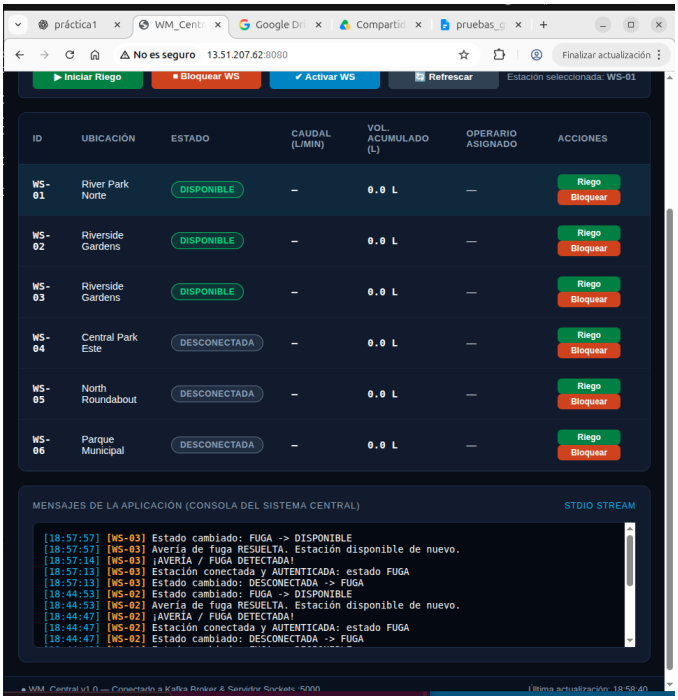


Figura 10. Panel de WM_Central con WS-01, WS-02 y WS-03 en estado DISPONIBLE.

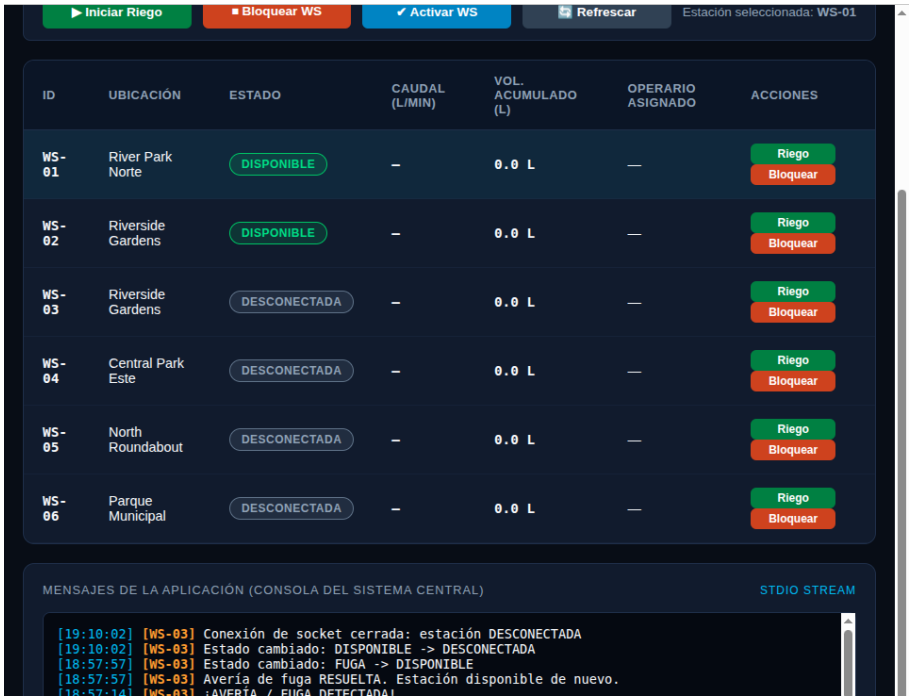


Figura 11. Panel de WM_Central tras la caída de WS-03: la estación pasa a DESCONECTADA mientras WS-01 y WS-02 siguen DISPONIBLE.

Prueba 4. Parametrización de los módulos

Objetivo: comprobar que los parámetros de ejecución de WM_WS y WM_FO pueden modificarse sin cambiar el código fuente ni reconstruir las imágenes Docker.

4.1. Parametrización de una estación. Utilizamos WS-03, desplegada en LLIBRE1-12, cuyo Monitor escuchaba inicialmente en el puerto 6003. Detuvimos los dos componentes y volvimos a iniciar la estación cambiando el puerto de comunicación de 6003 a 6004:

```
docker stop amazing_meninsky boring_allen
./docker_run.sh monitor 6004 13.51.207.62:5000 WS-03 --loc "Riverside Gardens"
./docker_run.sh engine 16.171.14.87:9092 localhost:6004
```

Durante el procedimiento mantuvimos en funcionamiento WM_Central, Kafka y las estaciones WS-01 y WS-02, sin modificar el código fuente ni reconstruir las imágenes.

Resultado: el Monitor de WS-03 arrancó correctamente en el puerto 6004 y se autenticó en WM_Central. El Engine consiguió conectarse al Monitor y recibió el identificador WS-03. Al consultar el listado desde FO-03, las tres estaciones aparecían como DISPONIBLE.

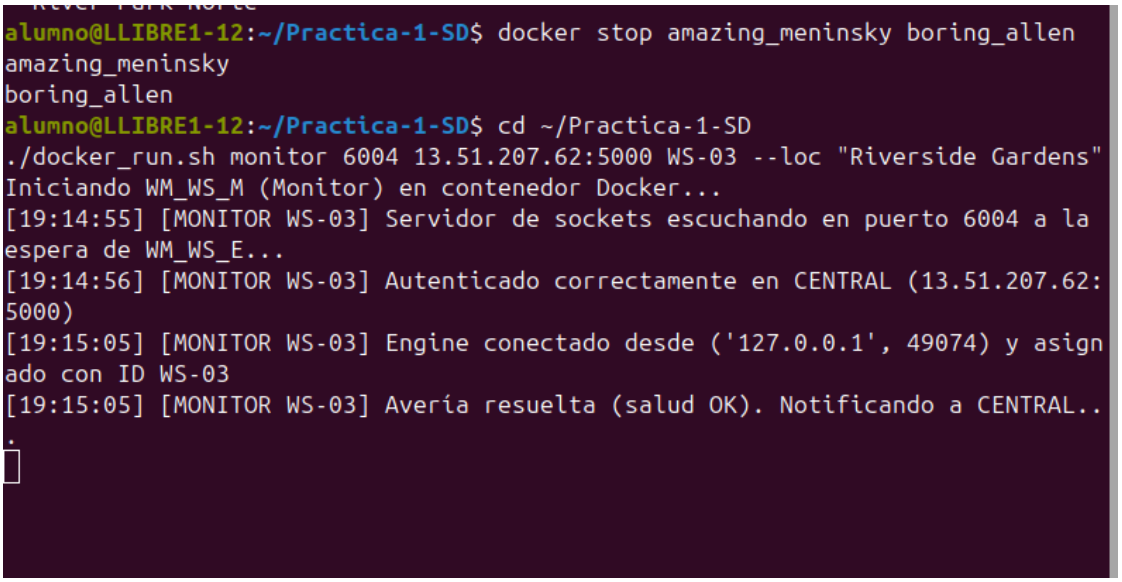


Figura 12. Parada de WS-03 con docker stop y nuevo arranque de su Monitor en el puerto 6004 mediante argumentos de ejecución.

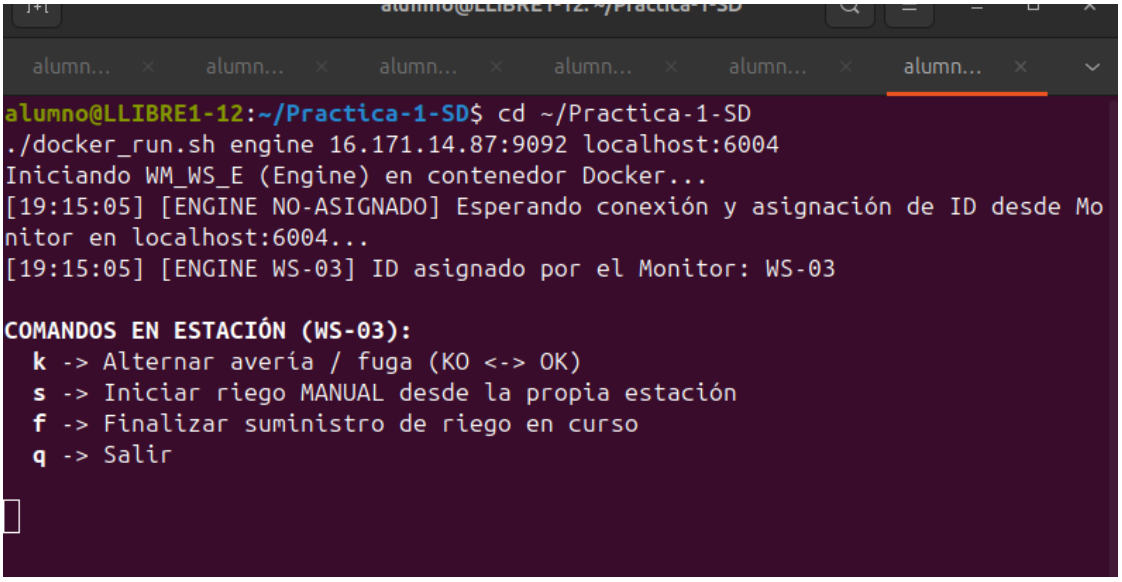


Figura 13. Engine de WS-03 conectado al Monitor en localhost:6004, que le asigna el identificador WS-03.

4.2. Parametrización del operador de campo. En LLIBRE1-11 iniciamos una nueva instancia de WM_FO identificada como FO-04 (`./docker_run.sh fo 16.171.14.87:9092 FO-04`). El comando establece la dirección y el puerto del broker Kafka, así como el identificador del operador.

Resultado: FO-04 se comunicó correctamente con el sistema y mostró las seis estaciones registradas: WS-01, WS-02 y WS-03 como DISPONIBLE y WS-04, WS-05 y WS-06 como DESCONECTADA. Estas comprobaciones demuestran que los parámetros de conexión, puertos e identificadores se pueden establecer desde la línea de comandos, sin modificar el código fuente.

Prueba 5. Funcionamiento de WM_Central y ejecución completa de un servicio de riego

Objetivo: comprobar que WM_Central permanece operativo, gestiona simultáneamente las estaciones conectadas y coordina correctamente una solicitud de riego desde un operador hasta su finalización.

5.1. Funcionamiento de WM_Central. Accedimos mediante SSH a la instancia AWS y comprobamos el estado del contenedor y los últimos mensajes del servidor, manteniendo conectadas varias estaciones y operadores:

```
sudo docker ps --filter name=wm_central
sudo docker logs --tail 40 wm_central
```

Resultado: el contenedor `wm_central` permanecía activo, con los puertos 5000 y 8080 publicados. En los registros observamos que Central mantenía actualizada la información de las seis estaciones registradas: WS-01, WS-02 y WS-03 como DISPONIBLE y WS-04, WS-05 y WS-06 como DESCONECTADA.

5.2. Ejecución completa de un servicio de riego. Utilizamos un despliegue distribuido con WM_Central y Kafka en AWS, el Monitor y el Engine de WS-01 en el portátil de Sofía y FO-01

en el portátil de Katia. Una vez comprobado que WS-01 aparecía como DISPONIBLE, iniciamos FO-01 (./docker_run.sh fo 16.171.14.87:9092 FO-01) y solicitamos un servicio de diez segundos (r WS-01 10).

Resultado: WM_Central recibió la solicitud, comprobó la disponibilidad de WS-01 y autorizó el riego. El Engine abrió la electroválvula y generó telemetría de caudal y volumen acumulado, que se recibió en la consola del operador. Al finalizar el tiempo solicitado, el Engine cerró automáticamente la electroválvula con motivo TIME, y FO-01 recibió el resumen:

```
Riego FINALIZADO (TIME): 1.35 L consumidos en 10 s
```

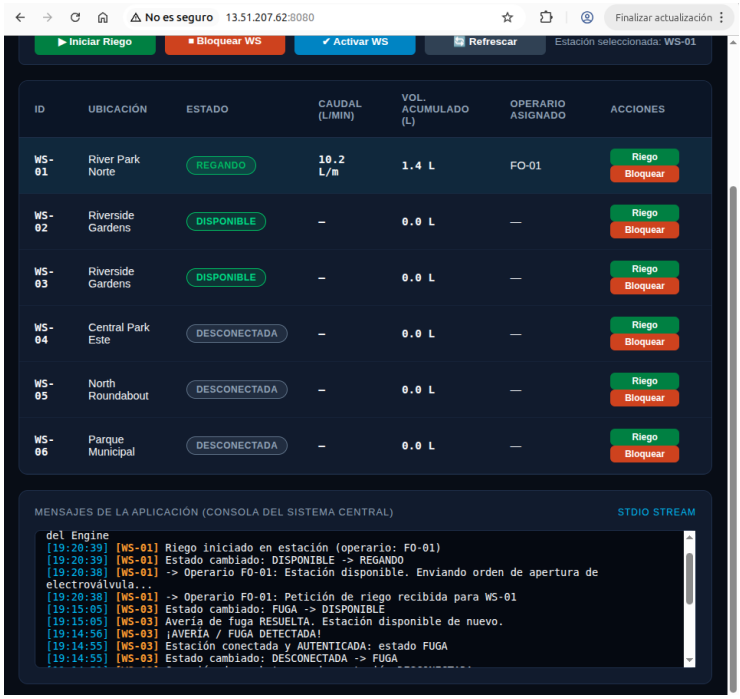


Figura 14. Panel de WM_Central durante un riego solicitado por FO-01: WS-01 en estado REGANDO, con caudal y volumen acumulado.

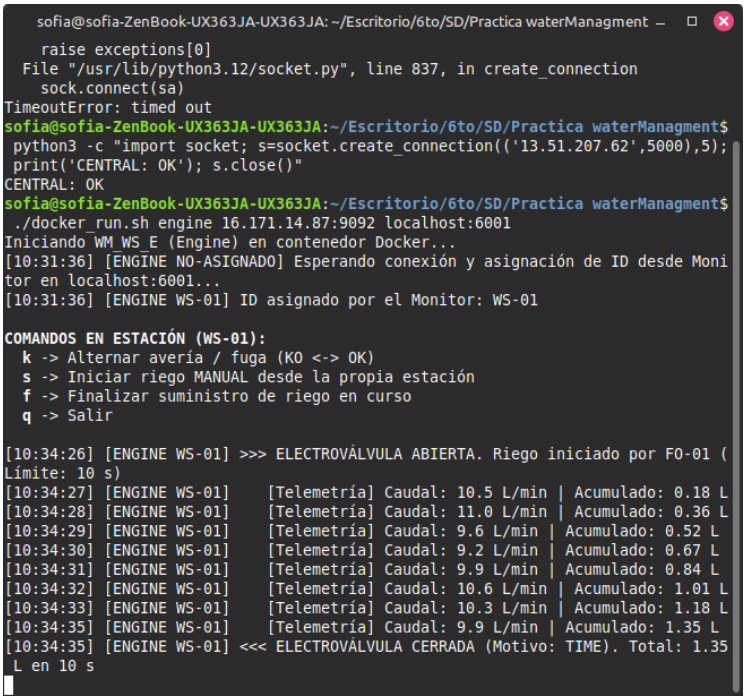


Figura 15. Engine de WS-01: apertura de la electroválvula, telemetría de caudal y volumen acumulado y cierre automático por TIME con 1,35 L en 10 s.

Durante las primeras comprobaciones se había detectado que determinados riegos registraban un segundo adicional. Se corrigió el control de duración del Engine y se realizaron ocho pruebas automáticas, todas satisfactorias. Posteriormente, la versión actualizada se desplegó y se verificó mediante esta ejecución real.

Prueba 6. Ejecución completa del sistema sin intervención

Objetivo: comprobar que los componentes de WaterManagement pueden coordinarse para ejecutar una secuencia de operaciones automáticamente, sin necesidad de introducir órdenes durante su desarrollo.

Procedimiento: utilizamos un despliegue distribuido mediante Docker. WM_Central y Kafka se encontraban en AWS, los módulos Monitor y Engine de WS-01 y WS-02 se ejecutaban en el portátil de Sofía, y FO-01 en el portátil de Katia. Una vez arrancados los componentes, comprobamos en el panel de WM_Central que ambas estaciones estaban DISPONIBLE. A continuación, utilizamos el modo de ejecución por lotes de WM_FO, proporcionando un fichero `operaciones.txt` con las siguientes solicitudes:

```
WS-01,10
WS-02,8
WS-01,5
```

El operador se inició mediante Docker:

```
docker run --rm -it --network=host \
  -v "$(pwd)/WM_FO/operaciones.txt:/app/operaciones.txt:ro" \
  wm_app python WM_FO/WM_FO.py \
```

```
16.171.14.87:9092 FO-01 \
--file operaciones.txt
```

Una vez iniciado el procesamiento del fichero, no fue necesario introducir órdenes adicionales. Los servicios se solicitaron y ejecutaron secuencialmente, con una espera automática de cuatro segundos entre ellos.

Resultado: los tres riegos se realizaron correctamente:

Estación	Duración	Volumen consumido	Finalización
WS-01	10 s	1,45 L	TIME
WS-02	8 s	0,96 L	TIME
WS-01	5 s	0,68 L	TIME

En las terminales se observó la recepción y autorización de las solicitudes, la apertura de las electroválvulas, el envío de telemetría y el cierre automático al alcanzar la duración establecida. Al terminar cada operación, FO-01 recibió el resumen correspondiente y el programa mostró el mensaje `Procesamiento del fichero por lotes completado con éxito`. Los tres servicios quedaron registrados en SQLite con los identificadores 11, 12 y 13, con la estación, el operario, las horas de inicio y fin, el volumen consumido, la duración y el motivo de finalización. Tras completar la secuencia, WS-01 y WS-02 aparecían nuevamente como DISPONIBLE en el panel.

Prueba 7. Recuperación de las estaciones registradas tras reiniciar WM_Central

Objetivo: comprobar que WM_Central consulta su base de datos SQLite al arrancar, recupera las estaciones registradas y conserva sus ubicaciones, mostrándolas como desconectadas mientras sus Monitors no estén conectados.

Procedimiento: la prueba se realizó con WM_Central desplegado en AWS mediante Docker y utilizando una base de datos SQLite persistente. En primer lugar, comprobamos que el contenedor utilizaba el archivo `/data/central.db`, almacenado en `/home/ubuntu/Practica-1-SD/data` mediante un volumen Docker. Detuvimos los Monitors y Engines de WS-01 y WS-02 y verificamos desde el panel web que las seis estaciones registradas aparecían como DESCONECTADA, conservando sus identificadores y ubicaciones. Después reiniciamos WM_Central, comprobamos de nuevo el panel y, finalmente, arrancamos únicamente el Monitor de WS-01:

```
sudo docker restart wm_central
./docker_run.sh monitor 6001 13.51.207.62:5000 WS-01 --loc "River Park Norte"
```

Resultado: tras reiniciar WM_Central, se recuperaron correctamente las seis estaciones registradas en SQLite:

Estación	Ubicación recuperada	Estado inicial
WS-01	River Park Norte	DESCONECTADA
WS-02	Riverside Gardens	DESCONECTADA
WS-03	Riverside Gardens	DESCONECTADA

Estación	Ubicación recuperada	Estado inicial
WS-04	Central Park Este	DESCONECTADA
WS-05	North Roundabout	DESCONECTADA
WS-06	Parque Municipal	DESCONECTADA

La información se conservó después del reinicio y no fue necesario registrar nuevamente las estaciones. Al iniciar el Monitor de WS-01, este se autenticó correctamente en Central y la estación cambió de DESCONECTADA a FUGA, ya que su Engine todavía no estaba conectado. Las otras cinco estaciones permanecieron desconectadas.

Prueba 8. Visualización de los estados de las estaciones en WM_Central

Objetivo: comprobar que el panel de WM_Central muestra la información de las estaciones y representa correctamente sus diferentes estados mediante los colores e indicadores establecidos en la guía de corrección.

Procedimiento: la prueba se realizó utilizando WM_Central desplegado en AWS y los módulos Monitor y Engine de WS-01 ejecutándose mediante Docker. Primero comprobamos el estado de las estaciones con sus Monitors desconectados. Posteriormente, arrancamos únicamente el Monitor de WS-01 y observamos cómo la estación pasaba a FUGA al no disponer todavía de su Engine. Después iniciamos el Engine para comprobar la transición al estado DISPONIBLE. Para verificar el estado de riego, iniciamos un servicio desde FO-01 y observamos el panel durante su ejecución, comprobando la actualización del caudal, el volumen acumulado y el indicador visual. A continuación, utilizamos la opción **Bloquear WS** para cambiar la estación al estado fuera de servicio y, finalmente, pulsamos **Activar WS** para devolverla a su estado operativo.

Resultado: se comprobaron los cinco estados establecidos:

Estado	Representación	Comportamiento observado
Disponible	Verde	WS-01 aparecía operativa tras conectar su Engine.
Regando	Verde parpadeante	El panel mostraba el riego en curso, el caudal y el volumen acumulado.
Fuga	Rojo	WS-01 aparecía averiada cuando su Monitor estaba conectado sin el Engine.
Fuera de servicio	Naranja	WS-01 mostraba la leyenda «FUERA DE SERVICIO» tras utilizar la opción de bloqueo.
Desconectada	Gris	Las estaciones sin conexión aparecían identificadas como desconectadas.

También se verificó que el panel mostraba el identificador, la ubicación, el estado y el volumen de agua de cada estación, así como la información asociada al servicio durante el riego. Los contadores superiores se actualizaron de acuerdo con los cambios de estado. Al finalizar, activamos WS-01 y comprobamos que volvía a aparecer como DISPONIBLE.

Prueba 9. Gestión de solicitudes, comunicación y conservación de transacciones

Objetivo: comprobar que WM_Central autoriza o deniega las solicitudes según el estado de la estación, comunica las respuestas a los operadores y conserva los resultados de los servicios sin duplicar transacciones.

9.1. Validación de solicitudes según la disponibilidad. Utilizamos FO-01 y las estaciones WS-01 y WS-02. Primero solicitamos un riego de cinco segundos en WS-02, que se encontraba desconectada. WM_Central recibió la petición, comprobó el estado de la estación y denegó el servicio; FO-01 recibió el mensaje de denegación y no se inició ningún riego. A continuación, tras verificar que WS-01 comunicaba correctamente el estado de su Engine, la estación pasó de FUGA a DISPONIBLE y solicitamos un riego de cinco segundos desde FO-01.

Resultado: Central autorizó la operación y el Engine abrió la electroválvula. Durante el servicio, el operador recibió los datos de caudal y volumen acumulado. El riego finalizó automáticamente con un consumo de 0,52 litros. SQLite almacenó la operación con identificador 15, asociada a WS-01 y FO-01, con duración de cinco segundos y motivo TIME.

9.2. Bloqueo y reactivación desde WM_Central. Utilizamos WS-01, disponible y conectada mediante su Monitor y Engine. Enviamos una orden de bloqueo desde el panel web: WS-01 pasó al estado FUERA DE SERVICIO (naranja) y el Engine recibió la orden. Al solicitar un riego de cinco segundos desde FO-01, Central lo denegó porque la estación estaba bloqueada. A continuación enviamos la orden de reactivación, el Engine confirmó su recepción y WS-01 volvió al estado DISPONIBLE.

Resultado: al solicitar un nuevo riego de cinco segundos, el servicio fue autorizado, la electroválvula se abrió y el riego finalizó automáticamente con un consumo de 0,48 litros.

9.3. Comunicación de mensajes y notificaciones. WM_Central utiliza Kafka para publicar las respuestas dirigidas a los operadores en el tópico fo_replies y las órdenes destinadas a las estaciones en ws_commands. Durante las pruebas, FO-01 recibió los mensajes correspondientes a la recepción de solicitudes, autorización y denegación de riegos, telemetría en tiempo real y resumen final. También comprobamos que el Engine de WS-01 recibía y ejecutaba las órdenes de bloqueo y reactivación enviadas desde Central.

Resultado: los mensajes y órdenes comprobados se transmitieron correctamente entre Central, FO y Engine, y sus efectos pudieron observarse en las interfaces correspondientes.

9.4. Conservación de transacciones y prevención de duplicados. Para comprobar la conservación de las transacciones, se incorporó un identificador único (transaction_id) a cada operación de riego. Este identificador evita que un mismo evento de finalización se registre varias veces en SQLite, incluso cuando Kafka entrega mensajes repetidos. También se modificó

el procesamiento para que Central registre la transacción antes de enviar el resumen al operario y confirme los mensajes de Kafka después de procesarlos correctamente. Se ejecutaron 14 pruebas automáticas en los dos ordenadores, incluyendo casos de duplicación de eventos, errores de escritura en SQLite, recuperación de operaciones interrumpidas y compatibilidad con bases de datos anteriores. Todas finalizaron correctamente. Posteriormente, desplegamos la versión actualizada de Central en AWS y comprobamos que se conservaban los 17 riegos registrados anteriormente.

Resultado: el sistema utiliza identificadores únicos para las operaciones y evita registrar varias veces el mismo resultado. La información de las operaciones anteriores se conservó después del despliegue actualizado.

Prueba 10. Control manual del riego desde el menú de la estación y desde WM_Central

Objetivo: comprobar que el riego puede iniciarse y detenerse desde el menú del Engine y que WM_Central también puede controlar directamente una operación.

10.1. Control desde el menú de la estación. En el menú del Engine de WS-01 introdujimos el comando `s`, que inició el riego sin necesidad de solicitarlo desde WM_FO. Comprobamos que la electroválvula se abría, que se generaba telemetría y que Central mostraba la estación en estado REGANDO. Posteriormente, introdujimos el comando `f` para detener el suministro: la electroválvula se cerró y WS-01 volvió al estado DISPONIBLE.

Resultado: SQLite registró el riego con identificador 21, operario `MANUAL`, volumen de 0,46 litros, duración de cuatro segundos y motivo `MANUAL`.

10.2. Control desde WM_Central. En el panel web desplegado en AWS seleccionamos WS-01, en estado DISPONIBLE, y solicitamos un riego de cinco segundos. Comprobamos que la estación pasaba a REGANDO, que el Engine abría la electroválvula y que se generaba telemetría durante el suministro. Transcurridos los cinco segundos, el riego finalizó automáticamente y WS-01 volvió a DISPONIBLE.

Resultado: SQLite registró la operación con identificador 23, operario `CENTRAL`, duración de cinco segundos, consumo de 0,66 litros y un identificador único de transacción. Estas comprobaciones muestran que el riego puede controlarse desde el menú de la estación y desde WM_Central, además de hacerlo desde WM_FO.

Prueba 11. Control individual y global de las estaciones desde WM_Central

Objetivo: comprobar que WM_Central permite bloquear, activar e iniciar riegos en una o varias estaciones, teniendo en cuenta el estado real de sus componentes.

11.1. Bloqueo de una estación durante un riego. Iniciamos un riego en WS-01 y, mientras estaba en curso, utilizamos la opción **Bloquear** del panel web. **Resultado:** el Engine cerró inmediatamente la electroválvula. El riego finalizó a los cinco segundos, con motivo `CENTRAL_BLOCK` y un volumen acumulado de 0,64 litros. La estación pasó a FUERA DE SERVICIO (naranja). Posteriormente, solicitamos un nuevo riego desde WM_FO y Central rechazó la solicitud porque la estación estaba fuera de servicio por orden central.

11.2. Reactivación de una estación bloqueada. Después de comprobar el bloqueo, utilizamos la opción **Activar** del panel web. **Resultado:** WS-01 cambió de FUERA DE SERVICIO a DISPONIBLE, y en la terminal del Engine comprobamos que se recibía correctamente la orden de reactivación.

11.3. Bloqueo global de las estaciones. Conectamos seis estaciones a WM_Central: WS-01 a WS-05 estaban DISPONIBLE, mientras que WS-06 permanecía en FUGA porque no tenía su Engine conectado. Desde Central enviamos la orden de bloqueo dirigida a todas las estaciones (`ALL`). **Resultado:** las seis estaciones pasaron al estado FUERA DE SERVICIO, representado en naranja en el panel web.

11.4. Activación global de las estaciones. Después del bloqueo global, enviamos desde WM_Central la orden ACTIVATE ALL. Durante una versión preliminar observamos que las seis estaciones pasaban a DISPONIBLE, incluso WS-06, que no disponía de Engine. Este comportamiento era incorrecto y motivó una corrección posterior. En la versión final, la activación global evalúa la situación real de cada estación: aquellas cuyo Monitor y Engine funcionan vuelven a DISPONIBLE, mientras que una estación sin Engine permanece en FUGA. Esta mejora se comprobó específicamente en las pruebas 11.6 y 11.7.

11.5. Riego global desde WM_Central. Mantuvimos seis estaciones registradas: WS-01 a WS-05 disponían de Monitor y Engine, mientras que WS-06 tenía únicamente Monitor. En una comprobación inicial, al solicitar un riego global de cinco segundos, las cinco primeras estaciones completaron sus operaciones y WS-06 no pudo abrir la válvula: Central esperó diez segundos hasta denegarla por ausencia de confirmación. Esta incidencia permitió comprobar la necesidad de validar el estado del Engine antes de autorizar el servicio. Tras la corrección, una estación en FUGA por ausencia del Engine no debe ser tratada como DISPONIBLE ni recibir una autorización de riego; la confirmación de inicio y su tiempo de espera se mantienen como protección adicional frente a fallos imprevistos.

11.6. Activación de una estación sin Engine. Arrancamos WS-06 únicamente con su Monitor, dejando el Engine desconectado; inicialmente aparecía en FUGA (rojo). Desde WM_Central enviamos una orden de bloqueo y comprobamos que pasaba a FUERA DE SERVICIO; posteriormente, enviamos la orden de activación. **Resultado:** la estación volvió a FUGA, en lugar de pasar incorrectamente a DISPONIBLE, ya que seguía sin tener Engine conectado.

11.7. Activación global con estaciones en diferentes estados. Preparamos seis estaciones en diferentes situaciones: WS-01 desconectada, WS-02, WS-04 y WS-05 disponibles, WS-03 fuera de servicio y WS-06 en fuga por no tener Engine conectado. Enviamos desde Central la orden `ACTIVATE ALL`. **Resultado:** WS-03 volvió a DISPONIBLE y las estaciones que ya estaban disponibles mantuvieron su estado. WS-01 permaneció desconectada y WS-06 continuó en FUGA. Se comprobó que la activación global se procesa individualmente y no convierte en disponibles las estaciones que no están operativas.

11.8. Persistencia del bloqueo ante averías del Engine. Bloqueamos WS-03 desde el panel web y verificamos que pasaba a FUERA DE SERVICIO. A continuación, simulamos una avería desde el Engine pulsando k; la estación permaneció en naranja. Después volvimos a pulsar k para recuperar el Engine. **Resultado:** WS-03 continuó bloqueada después de recuperarse el Engine y únicamente volvió a DISPONIBLE cuando enviamos una orden explícita de activación desde Central.

11.9. Persistencia del bloqueo tras reiniciar el Engine. Bloqueamos WS-03 y después detuvimos su Engine. Tras comprobar la incidencia, reiniciamos el Engine y verificamos que el Monitor detectaba su reconexión. Finalmente, intentamos iniciar un riego manual desde el propio Engine. **Resultado:** la estación continuó bloqueada y rechazó el riego manual; la reconexión del Engine no eliminó la orden de bloqueo recibida desde Central.

Prueba 12. Autenticación del Monitor, supervisión del Engine y detección de fugas

Objetivo: comprobar que el Monitor se autentica en WM_Central, supervisa periódicamente el Engine y comunica las incidencias y recuperaciones.

12.1. Autenticación y comprobación de salud. Arrancamos el Monitor y el Engine de WS-02 y comprobamos que el Monitor se autenticaba correctamente en WM_Central. Revisamos el bucle de monitorización y verificamos que el Monitor enviaba periódicamente mensajes HEALTH al Engine, con una espera de un segundo entre comprobaciones. **Resultado:** el Monitor pudo comprobar el estado del Engine y detectar las respuestas de funcionamiento o avería.

12.2. Detección de fuga por desconexión del Engine. Detuvimos el Engine de WS-02 mientras su Monitor continuaba funcionando. **Resultado:** el Monitor detectó que el Engine había dejado de responder y notificó la incidencia a WM_Central con el motivo NO_ENGINE. WS-02 pasó al estado FUGA (rojo) y permaneció así mientras el Engine estuvo desconectado.

12.3. Detección de avería del Engine. Para simular una avería del Engine de WS-02, pulsamos k durante su ejecución; el Engine pasó a responder KO a las comprobaciones de salud. **Resultado:** el Monitor detectó la incidencia, notificó la fuga a WM_Central y WS-02 cambió de DISPONIBLE a FUGA en el panel web.

12.4. Detección de una fuga durante un riego. Solicitamos desde FO-01 un riego de treinta segundos sobre WS-02 y, mientras estaba en curso, simulamos una avería en el Engine pulsando k. **Resultado:** la electroválvula se cerró inmediatamente y el riego finalizó con motivo LEAK, después de cinco segundos y con un consumo de 0,66 litros. El operario recibió la notificación y SQLite conservó el resultado de la operación.

12.5. Recuperación automática de una fuga. Después de simular la fuga, volvimos a pulsar k en el Engine para eliminar la avería. **Resultado:** el Monitor detectó la recuperación y WM_Central registró la resolución de la incidencia. La estación pasó automáticamente de FUGA a DISPONIBLE, sin necesidad de enviar una orden de activación desde Central.

12.6. Visualización de incidencias. Observamos los mensajes del Monitor de WS-02 durante la simulación y posterior resolución de la avería. **Resultado:** el Monitor mostró tanto la detección de la fuga como su resolución y notificó los cambios a WM_Central. En el panel web se observaron las transiciones entre FUGA y DISPONIBLE, junto con los eventos correspondientes.

Prueba 13. Consulta global de estaciones y aislamiento de mensajes entre operadores

Objetivo: comprobar que todos los operadores pueden consultar la red completa y que cada uno recibe únicamente los mensajes correspondientes a sus propios servicios.

13.1. Consulta global de estaciones. Iniciamos FO-01 y FO-02 en ordenadores diferentes y ejecutamos el comando l desde ambos. **Resultado:** los dos operadores pudieron consultar el listado completo de estaciones, incluyendo sus ubicaciones y estados actuales, independientemente de si tenían un riego asignado.

13.2. Aislamiento de mensajes entre operadores. Mantuvimos conectados FO-01 y FO-02 simultáneamente. Desde FO-01 solicitamos un riego de quince segundos sobre WS-02. **Resultado:** FO-01 recibió los mensajes de autorización, la telemetría durante el riego y el resumen final, mientras que FO-02 no recibió las notificaciones de esa operación.

13.3. Aislamiento de mensajes durante riegos simultáneos. FO-01 solicitó un riego de veinte segundos sobre WS-02 y, mientras estaba en curso, FO-02 solicitó otro de veinte segundos sobre WS-03. **Resultado:** ambos riegos se ejecutaron correctamente. Cada operador recibió únicamente los mensajes correspondientes a su estación, incluyendo la autorización, la telemetría y el resumen final.

Prueba 14. Concurrencia de operaciones en WM_Central

Objetivo: comprobar que WM_Central puede atender distintas operaciones simultáneamente sin bloquear las consultas, los servicios de riego ni el panel de supervisión.

14.1. Concurrencia de peticiones en WM_Central. Iniciamos desde FO-01 un riego de treinta segundos en WS-02 y, mientras estaba en curso, utilizamos FO-02 desde otro ordenador para consultar el listado de estaciones mediante l. **Resultado:** Central respondió correctamente a la consulta mientras WS-02 continuaba regando, sin que una operación interrumpiera la otra.

14.2. Concurrencia entre Monitor, Engine y Central. Iniciamos un riego manual desde el Engine de WS-02 y, mientras ejecutaba el riego y enviaba telemetría, utilizamos FO-01 para consultar las estaciones disponibles. **Resultado:** Central respondió correctamente y mostró WS-02 en estado REGANDO. Al detener el riego manual, registró su finalización y devolvió la estación al estado DISPONIBLE.

14.3. Disponibilidad del servidor durante operaciones simultáneas. Con dos operadores conectados desde ordenadores diferentes, FO-01 solicitó un riego de veinte segundos sobre WS-02 y FO-02 solicitó otro sobre WS-03. Mientras ambos riegos estaban en curso, comprobamos que el panel web de WM_Central seguía funcionando y mostraba las dos estaciones en estado REGANDO. **Resultado:** los dos riegos finalizaron correctamente y cada operador recibió su

resumen correspondiente. El panel de monitorización permaneció disponible durante las operaciones simultáneas.

Prueba 15. Aislamiento de fallos en componentes independientes

Objetivo: comprobar que la caída de un componente de una estación o de un operador no impide que el resto del sistema continúe funcionando.

15.1. Aislamiento del fallo de un Engine. Detuvimos el Engine de WS-02, manteniendo su Monitor conectado. **Resultado:** WS-02 pasó a FUGA, mientras que WS-03 permaneció DISPONIBLE. Solicitamos desde FO-01 un riego de cinco segundos sobre WS-03 y se completó correctamente.

15.2. Aislamiento del fallo de un Monitor. Detuvimos el Monitor de WS-02 mientras el resto de los componentes continuaban funcionando. **Resultado:** WS-02 pasó a DESCONECTADA, mientras que WS-03 permaneció DISPONIBLE. El riego de cinco segundos solicitado sobre WS-03 se completó correctamente.

15.3. Aislamiento del fallo de un operador. Cerramos FO-01 mientras FO-02 permanecía conectado desde otro ordenador. A continuación, solicitamos desde FO-02 un riego de cinco segundos sobre WS-03. **Resultado:** el riego se ejecutó correctamente y FO-02 recibió los mensajes correspondientes. La desconexión de FO-01 no afectó al otro operador.

15.4. Caída y recuperación de WM_Central. Detuvimos el contenedor Docker de WM_Central en AWS mientras el resto de los componentes continuaban funcionando. **Resultado:** el Monitor de WS-03 detectó la pérdida de conexión e intentó reconectarse automáticamente. El Engine mostró un aviso indicando que Central no estaba disponible y rechazó nuevas solicitudes de riego. Posteriormente reiniciamos WM_Central: el Monitor recuperó la conexión sin necesidad de reiniciarlo y FO-02 pudo solicitar un nuevo riego de cinco segundos en WS-03, que se completó correctamente.

15.5. Caída y recuperación del broker Kafka. Durante las primeras pruebas, al detener Kafka en AWS observamos que las publicaciones síncronas podían bloquear la atención de las comprobaciones HEALTH del Engine. Esto originaba falsas alertas NO_ENGINE y MONITOR_DOWN, e interrumpía riegos manuales. Se corrigió separando las operaciones de Kafka de la supervisión TCP, haciendo independiente el temporizador de riego y protegiendo el envío y la recuperación de eventos pendientes. En la validación final, con Kafka detenido, FO-03 mostró un error controlado al intentar enviar una solicitud. Después iniciamos un riego manual en WS-03: el Engine notificó el timeout de Kafka, pero continuó regando durante quince segundos sin falsas desconexiones del Monitor. Al detener manualmente el proceso del Engine con Ctrl+C, la válvula se cerró con 2,21 L; el NO_ENGINE posterior correspondió a esa parada, no a Kafka. Tras restablecer el broker y reiniciar el Engine, WS-03 recuperó la disponibilidad. Finalmente, FO-03 solicitó un riego de cinco segundos, que terminó por TIME con 0,73 L y resumen recibido. La prueba confirmó el aislamiento entre la caída de Kafka y el canal de supervisión Monitor–Engine, así como la recuperación del servicio.

Prueba 16. Caída de componentes durante la ejecución de un riego

Objetivo: comprobar el comportamiento del sistema cuando se produce una caída inesperada del Monitor, del Engine, del Front Office o de WM_Central durante un servicio de riego.

16.1. Caída del Monitor durante un riego. Iniciamos desde FO-02 un riego de treinta segundos en WS-03 y, mientras estaba en curso, detuvimos abruptamente su Monitor mediante Ctrl+C. **Resultado:** WM_Central marcó la estación como DESCONECTADA y el Engine detectó la pérdida de conexión, cerrando inmediatamente la electroválvula con motivo MONITOR_DOWN. El riego se interrumpió después de ocho segundos, con un consumo acumulado de 1,17 litros. FO-02 recibió un aviso indicando la desconexión y el volumen suministrado. También intentamos iniciar un nuevo riego manual desde el Engine y comprobamos que lo rechazaba por falta de conexión con el Monitor.

16.2. Caída del Engine durante un riego. Iniciamos desde FO-02 un riego de treinta segundos en WS-03 y detuvimos el Engine mediante Ctrl+C mientras estaba regando. **Resultado:** el Monitor detectó la pérdida de conexión con el Engine y notificó la avería a WM_Central, que marcó WS-03 como FUGA. El operario recibió un resumen indicando que el riego había finalizado por avería (LEAK), con un consumo de 0,84 litros en seis segundos. Posteriormente reiniciamos únicamente el Engine, que se reconectó al Monitor; se resolvió la avería y WS-03 volvió a estar disponible. SQLite conservó los datos correctos del riego interrumpido y únicamente existía un registro para su identificador de transacción.

16.3. Caída del Front Office durante un riego. Iniciamos desde FO-02 un riego de treinta segundos en WS-03 y cerramos abruptamente la aplicación del operador mediante Ctrl+C mientras el servicio estaba en curso. **Resultado:** el Engine continuó regando con normalidad y el servicio finalizó al alcanzar los treinta segundos. Posteriormente volvimos a iniciar FO-02 utilizando el mismo identificador y comprobamos que recibía el resultado del riego realizado durante su ausencia.

16.4. Caída de WM_Central durante un riego. Iniciamos desde FO-02 un riego de treinta segundos en WS-03 y detuvimos el contenedor de Central mientras el servicio estaba en curso. **Resultado:** el Engine detectó la pérdida de conexión con Central, pero continuó el riego hasta completar los treinta segundos. La electroválvula se cerró con un consumo total de 4,33 litros, y el Monitor detectó la desconexión e inició los intentos de reconexión. Mientras Central permanecía detenida, solicitamos un nuevo riego desde FO-02: la solicitud no fue autorizada y se mostró un mensaje indicando que era imposible conectar con WM_Central. Después reiniciamos Central y comprobamos que el Monitor y el Engine recuperaban automáticamente la conexión. El Engine notificó el resultado del riego anterior y, finalmente, solicitamos un nuevo riego de cinco segundos, que se completó correctamente con un consumo de 0,65 litros.

Prueba 17. Estados del Monitor y del Engine

Objetivo: comprobar que WM_Central representa correctamente el estado de las estaciones según la disponibilidad de su Monitor y su Engine.

Procedimiento: verificamos las cuatro combinaciones posibles de funcionamiento de ambos componentes: (1) Monitor y Engine operativos; (2) Monitor operativo y Engine no operativo; (3) Monitor no operativo y Engine operativo; (4) Monitor y Engine no operativos.

Resultado:

Monitor	Engine	Estado mostrado
Operativo	Operativo	Disponible (verde)
Operativo	No operativo	Fuga (rojo)
No operativo	Operativo	Desconectada (gris)
No operativo	No operativo	Desconectada (gris)

Con ambos componentes activos, WS-03 aparecía disponible. Al detener únicamente el Engine, Central marcaba la estación como averiada. Cuando detuvimos el Monitor mientras el Engine seguía funcionando, la estación pasó a desconectada. Finalmente, al detener ambos componentes, WS-03 también apareció desconectada. Se comprobó que la ausencia de comunicación con el Monitor tiene prioridad sobre el estado del Engine.

Prueba 18. Recuperación del sistema e incorporación de nuevas instancias

Objetivo: comprobar que los componentes pueden recuperarse reiniciando únicamente el módulo afectado y que es posible incorporar nuevos operadores y estaciones sin interrumpir el funcionamiento de las instancias existentes.

Procedimiento y resultados: aprovechamos las pruebas anteriores para comprobar la recuperación de distintos componentes. Los Monitors, Engines y operadores recuperaron sus conexiones sin necesidad de reiniciar el resto del sistema. También comprobamos que WM_Central y Kafka podían restablecerse sin reiniciar las estaciones. Después incorporamos un nuevo operador, FO-03, mientras el sistema continuaba funcionando; comprobamos que podía consultar el listado actualizado de estaciones y que FO-02 seguía funcionando normalmente.

Por último, registramos dinámicamente una nueva estación, WS-07, con ubicación Central Park Oeste, sin reiniciar Central ni los demás componentes. El registro se realizó mediante el mecanismo de administración de WM_Central; la operación devolvió HTTP 200 y confirmó la creación de la estación. Primero conectamos su Monitor y comprobamos que aparecía en estado FUGA al no tener todavía el Engine conectado; al iniciar el Engine, la estación pasó a DISPONIBLE. Finalmente, solicitamos desde FO-03 un riego de cinco segundos en WS-07.

Resultado: el riego se completó correctamente, con un consumo aproximado de 0,69 litros. Se comprobó que el sistema permite recuperar componentes e incorporar nuevas instancias sin afectar al funcionamiento de las existentes.

Prueba 19. Gestión controlada de errores ante caídas de componentes

Objetivo: comprobar que los fallos de comunicación o la caída de componentes se notifican de forma controlada en las interfaces correspondientes, sin cierres inesperados ni errores sin gestionar.

Procedimiento: se revisaron los mensajes de error mostrados por Central, el Monitor, el Engine y los operarios al provocar las interrupciones de las pruebas 15 y 16. Se completó la comprobación con una caída específica de Kafka y un riego manual en WS-03 (prueba 19.2).

1. Caída de WM_Central: comprobación de errores de conexión y denegación de nuevos riegos.
2. Caída del Monitor: detección de pérdida de enlace, aviso de avería y cierre seguro de la válvula.
3. Caída del Engine: detección de NO_ENGINE en el Monitor y comunicación de la incidencia a Central.
4. Caída de un FO: el servicio en curso continúa y el resultado se recupera tras reiniciar el operario.
5. Caída de Kafka: el FO informa de que no puede publicar la solicitud y conserva la ejecución interactiva.
6. Riego manual con Kafka detenido: el Engine informa del timeout sin provocar falsas desconexiones TCP.
7. Restablecimiento de los servicios: recuperación de las conexiones y ejecución de un nuevo riego completo.

Resultado: los errores de conexión se comunicaron mediante mensajes comprensibles, como «Imposible conectar con WM_Central» o «No se pudo enviar la solicitud al broker Kafka». En la prueba 19.2, con Kafka detenido, el Engine mantuvo el enlace TCP con su Monitor durante un riego manual de quince segundos. El NO_ENGINE observado después se debió a la parada voluntaria del Engine mediante Ctrl+C. Tras reiniciar Kafka y el Engine, FO-03 completó un riego de cinco segundos sobre WS-03, con cierre TIME y 0,73 L consumidos. No se observaron falsas averías provocadas únicamente por el fallo de Kafka.

Prueba 20. Memoria del proyecto

Objetivo: presentar la documentación del proyecto y describir su arquitectura, implementación, despliegue, funcionamiento y resultados.

Contenido: la memoria recoge la descripción general del sistema WaterManagement, la organización de sus módulos, las tecnologías utilizadas y las decisiones de diseño adoptadas. También documenta la arquitectura distribuida, la comunicación entre los componentes, el almacenamiento de información, los procedimientos de ejecución y las pruebas realizadas.

Resultado: la memoria (este documento) acompaña a la implementación como documentación técnica del proyecto y permite comprender la estructura del sistema, sus funcionalidades y el trabajo desarrollado.

Prueba 21. Interfaz de usuario

Objetivo: comprobar que las interfaces de WM_Central, WM_FO y los componentes de las estaciones permiten supervisar y controlar el sistema de forma clara.

Procedimiento: durante las pruebas de funcionamiento se utilizaron las interfaces de los distintos módulos:

- El dashboard de WM_Central para consultar estaciones, ubicaciones, estados y operaciones.
- La consola de WM_FO para consultar estaciones, solicitar riegos y recibir notificaciones.
- El menú del Engine para iniciar y detener riegos manualmente y simular incidencias.

Se comprobaron los cambios visuales de estado, la información de telemetría y los mensajes de autorización, denegación y finalización de los servicios.

Resultado: las interfaces permitieron consultar el estado de las estaciones y seguir el desarrollo de los servicios de riego. El dashboard mostró los estados mediante los colores establecidos y presentó información sobre el caudal y el volumen consumido. Las consolas de los operadores y de las estaciones mostraron las operaciones y sus resultados.

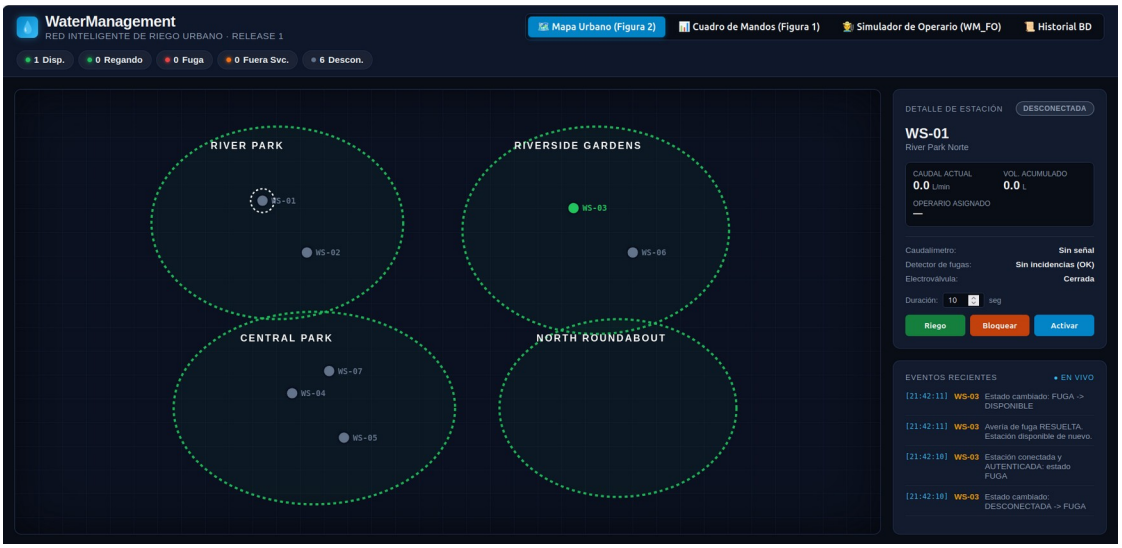


Figura 16. Mapa urbano del panel de WM_Central: estaciones agrupadas por zonas, con WS-03 DISPONIBLE (verde) y el resto DESCONECTADA (gris), junto al detalle de la estación seleccionada y los eventos recientes.

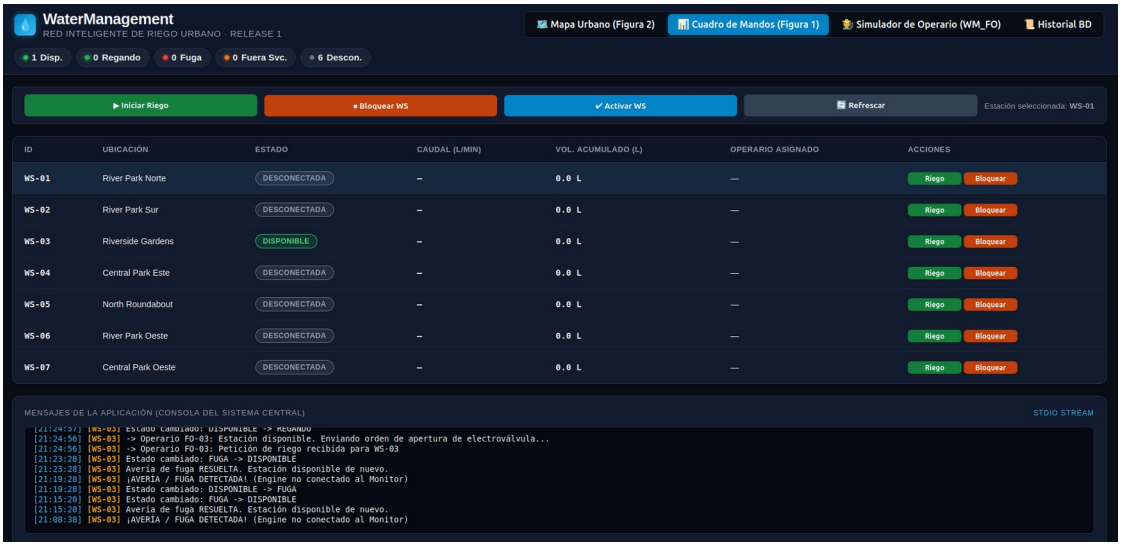


Figura 17. Cuadro de mandos de WM_Central: identificador, ubicación, estado, caudal, volumen acumulado y operario de cada estación, con los botones de control y la consola de mensajes del sistema.

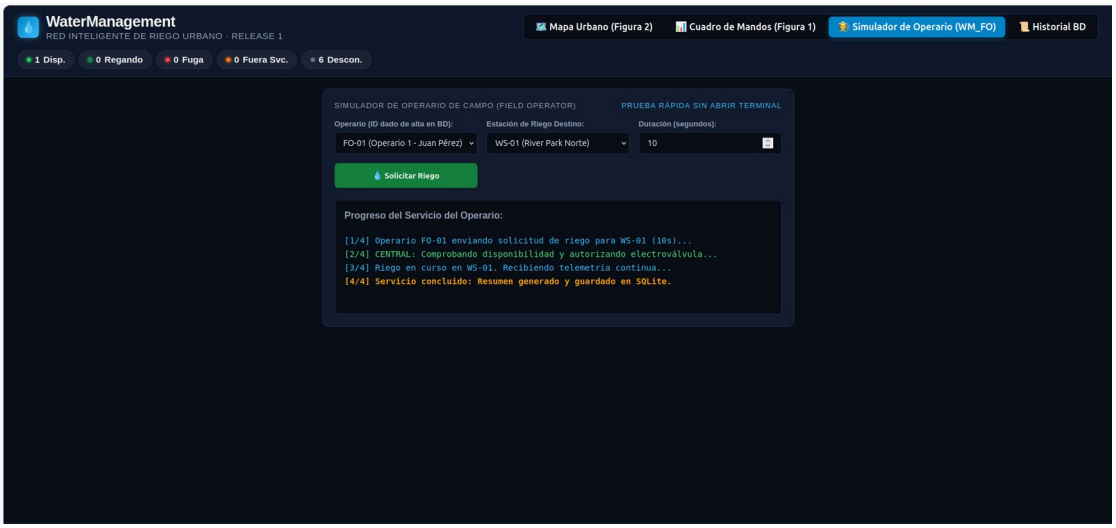


Figura 18. Simulador de operario del panel web: solicitud de un riego de 10 s de FO-01 sobre WS-01 y progreso del servicio hasta su registro en SQLite.

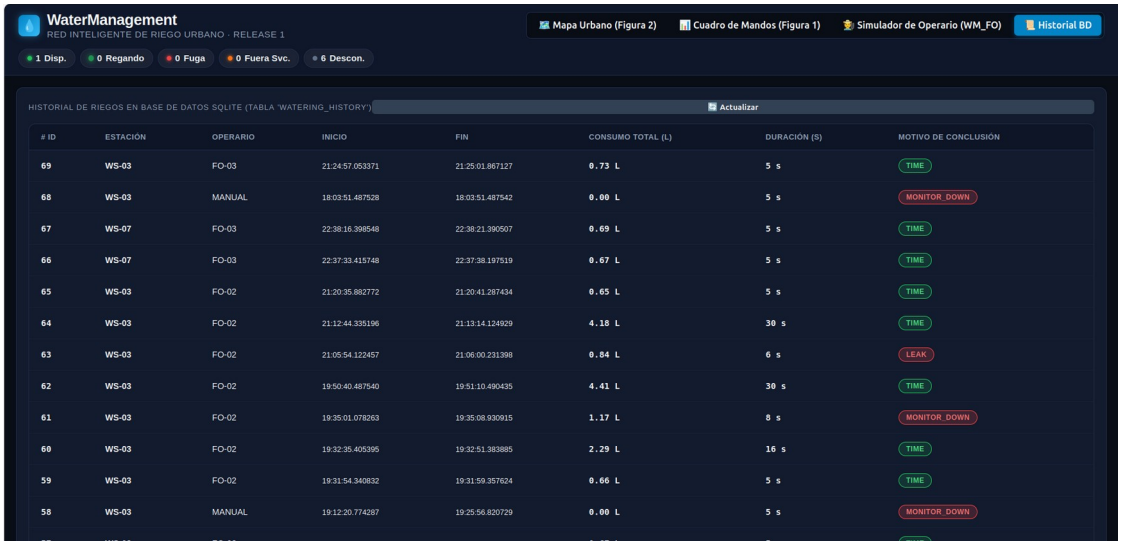


Figura 19. Historial de riegos del panel web, leído de la base de datos SQLite: estación, operario, inicio, fin, consumo, duración y motivo de conclusión (TIME, LEAK, MONITOR_DOWN).

La numeración 1–22 corresponde a los criterios de evaluación de la guía, mientras que las 26 pruebas automatizadas son comprobaciones de software independientes de esa numeración.

Prueba 22. Funcionalidades adicionales y validación global

Objetivo: recoger las funcionalidades adicionales incorporadas al proyecto y comprobar su integración con el funcionamiento general del sistema.

Procedimiento: durante el desarrollo y las pruebas se incorporaron y validaron las siguientes mejoras:

1. Registro dinámico de nuevas estaciones, sin necesidad de modificar manualmente la lista de identificadores ni reiniciar WM_Central.
2. Identificación única de las operaciones de riego mediante `transaction_id`.
3. Prevención de duplicados en el registro de resultados.
4. Recuperación de operaciones y conservación de sus resultados ante interrupciones.
5. Control del tiempo de riego para evitar registrar duraciones superiores a las solicitadas.
6. Recuperación de los módulos después de caídas individuales.
7. Ejecución simultánea de operadores y estaciones.

Además, se ejecutaron las pruebas automatizadas de los módulos de transacciones y flujo de riego:

```
python3 -m unittest -v test_transaction_duplicates test_watering_flow
```

Resultado: la última batería automatizada ejecutada sobre la versión corregida finalizó con 26 pruebas satisfactorias (OK), según los registros de ejecución: 19 de test_watering_flow.py y 7 de test_transaction_duplicates.py. Se incluyeron pruebas de temporización independiente de Kafka, respuesta HEALTH no bloqueante, bloqueo inmediato de estaciones, conservación de eventos concurrentes, control del orden de publicación y recuperación de FINISHED. Adicionalmente se validó el aislamiento de Kafka en AWS y un nuevo riego real de cinco segundos desde FO-03 sobre WS-03, que finalizó por TIME con 0,73 L.

8. Conclusiones

El desarrollo de la práctica WaterManagement nos ha permitido aplicar sobre un caso práctico completo los principios de diseño y construcción de sistemas distribuidos estudiados en la asignatura. La combinación de Sockets TCP (con empaquetado de tramas `<STX><DATA><ETX><LRC>` y verificación por XOR) para las labores de registro y control de salud, junto con Apache Kafka para la transmisión asíncrona de órdenes y telemetría, ha demostrado ser una solución muy adecuada: aprovecha la inmediatez de una conexión orientada a flujo allí donde es necesario detectar caídas de nodos, y la flexibilidad del modelo publicador-suscriptor allí donde interesa desacoplar a los clientes del núcleo central.

Así mismo, el proceso de pruebas sobre contenedores Docker puso de manifiesto la importancia de no asumir que todos los procesos de un sistema distribuido están siempre activos al mismo tiempo. La detección de los fallos iniciales cuando el proceso Engine estaba apagado nos llevó a implementar mecanismos de sincronización más rigurosos (como el estado inicial en avería hasta verificar la salud del motor y la confirmación en dos fases con timeout al iniciar cada riego), logrando finalmente un sistema resiliente, coherente en su base de datos SQLite y capaz de recuperarse ante fallos parciales de cualquiera de sus componentes.

Las pruebas realizadas permiten validar los principales aspectos funcionales y distribuidos de WaterManagement:

1. **Despliegue modular:** WM_Central se ejecuta en AWS mediante Docker y los módulos WM_WS y WM_FO pueden iniciarse como instancias independientes.
2. **Parametrización:** los identificadores, puertos y parámetros de conexión pueden establecerse desde los argumentos de ejecución.
3. **Gestión de estaciones:** el sistema permite registrar estaciones, consultar sus estados, bloquearlas, activarlas y gestionar solicitudes individuales y globales.
4. **Control del riego:** las operaciones incluyen autorización, ejecución, telemetría, cierre de válvula y comunicación del resultado.
5. **Concurrencia:** se pueden ejecutar servicios simultáneos y realizar consultas mientras otros servicios permanecen activos.
6. **Aislamiento:** los mensajes de cada servicio se entregan al operador que ha realizado la solicitud.
7. **Supervisión:** el Monitor comprueba periódicamente el estado del Engine y comunica las incidencias a WM_Central.
8. **Tolerancia a fallos:** el sistema detecta incidencias en el Monitor, el Engine, el Front Office, WM_Central y Kafka, y aplica el comportamiento correspondiente a cada situación.
9. **Persistencia:** SQLite conserva la información de las estaciones y los resultados de las operaciones.
10. **Recuperación:** los componentes pueden restablecer su funcionamiento después de una caída sin necesidad de reiniciar todo el sistema.
11. **Validación automatizada:** las pruebas unitarias ejecutadas finalizaron correctamente.

En conjunto, WaterManagement proporciona una arquitectura distribuida y modular capaz de gestionar múltiples estaciones de riego, atender solicitudes concurrentes, supervisar sus componentes y mantener un comportamiento controlado ante fallos de comunicación o ejecución.